

说明：全部源程序共 264 页，按受理要求提交前 30 页与后 30 页（本页为前 30 页之第 1 页）。

```
# =====
# 文件: lawgate/__init__.py
# =====
# -*- coding: utf-8 -*-
"""CA-LegalGate (律核) ——三通道异构法律问答路由系统。

在任何 torch / transformers 导入之前统一修正本机环境（幂等，见 ``lawgate.env_setup``）：
    * Anaconda MKL 与 torch 同时链接 libiomp5md.dll → ``KMP_DUPLICATE_LIB_OK``；
    * 远程代码模型的动态模块缓存指到仓库内（``HF_MODULES_CACHE``），
      否则会写到全局缓存目录而不可写/不可复现。
"""

from __future__ import annotations

from lawgate.env_setup import apply as _apply_env

_apply_env()

__version__ = "0.1.0"
__all__ = ["__version__"]

# =====
# 文件: lawgate/api/__init__.py
# =====
# -*- coding: utf-8 -*-
"""演示系统层：FastAPI 服务与 Gradio 双栏界面。"""

# =====
# 文件: lawgate/api/app.py
# =====
# -*- coding: utf-8 -*-
"""演示系统 • FastAPI 服务（手册 S7.1）。

启动：



```
uvicorn lawgate.api.app:app --host 127.0.0.1 --port 8000
```



或：



```
python -m lawgate.api.app --port 8000
```



接口：



|                   |                           |
|-------------------|---------------------------|
| GET /health       | 健康检查 + 后端溯源               |
| POST /chat        | 单轮/多轮问答，返回 answer + trace |
| POST /chat/stream | 同上，但 **SSE 流式**返回（见下）     |
| GET /trace/schema | 返回 trace 字段说明（答辩演示用）      |
| POST /verify_case | 单独调用案号核验                  |
| POST /temporal    | 单独调用时效状态机                 |


```

``/chat/stream`` 的帧协议（每帧都是 ``event: <名>`` + ``data: <JSON>``）：

```
event: stage      {"stage": "channel_b|retrieval|generate", "channel": "...", "u": "...", ...}
event: delta      {"text": "增量文本"}
event: done       {"answer": "权威全文", "trace": {...}, "provision": ...}
event: error      {"message": "异常类型: 异常信息"}
```

约定：``delta`` 只是**增量**，首尾空白可能与全文略有出入；客户端必须用 ``done.answer`` 收口（不能只靠拼接 delta）。答案主体与 ``/chat`` 完全同源——两条路径共用 router 的同一份决策逻辑，trace 字段一致。

为什么端点是**同步 def** 而不是 ``async def``：路由 + 生成是 CPU 密集的阻塞调用（1.5B CPU 推理十几秒），放进事件循环会卡死整个服务。Starlette 对同步生成器会自动用 ``iterate\_in\_threadpool`` 逐帧取数，事件循环始终空闲（本项目 D-系列教训）。

设计说明：``ROUTER`` 为进程内单例、惰性初始化，避免导入即加载 1GB 权重；模型与向量库加载失败时退回 ExtractiveLLM（响应中 ``trace.llm\_backend`` 会写明）。
"""

```
from __future__ import annotations

import json as _json
import os
import threading
import time
from pathlib import Path
from typing import Any

os.environ.setdefault("KMP_DUPLICATE_LIB_OK", "TRUE")
os.environ.setdefault("TOKENIZERS_PARALLELISM", "false")

from fastapi import FastAPI # noqa: E402
from fastapi.responses import StreamingResponse # noqa: E402
from pydantic import BaseModel, Field # noqa: E402

from lawgate.config import get_settings # noqa: E402

app = FastAPI(title="CA-LegalGate 律核",
              description="三通道异构法律问答路由系统（免训练）",
              version="0.1.0")

_ROUTER: Any = None
_LOAD_ERROR: str | None = None
_STARTED = time.time()
_ROUTER_LOCK = threading.Lock()

def get_router():
    """惰性单例：首次请求时才加载模型、向量库与通道 B。

    加锁防止并发首请求重复执行 build_context()（会加载 1GB+ 的模型权重）。
    """
    global _ROUTER, _LOAD_ERROR
```

```

if _ROUTER is None:
    with _ROUTER_LOCK:
        if _ROUTER is None:
            try:
                from lawgate.eval.run_exp import build_context

                ctx = build_context()
                _ROUTER = ctx.router
                _LOAD_ERROR = None
            except Exception as exc: # noqa: BLE001
                _LOAD_ERROR = f"{type(exc).__name__}: {exc}"
            raise
    return _ROUTER

class Turn(BaseModel):
    query: str
    answer: str | None = None
    # D26-4: **必须**保留 trace. detect_intent 的槽位继承只认"上一轮 trace.channel=='B'"
    # 的轮次 (D10/D12 的设计: 只继承经通道 B 事实确认的槽位)。
    # 原先这里只声明 query/answer, pydantic 默认丢弃未声明字段 → UI 明明把 trace
    # 塞进了 history, 却在 API 边界被丢掉 → 多轮继承在 /chat 上恒不触发。
    trace: dict | None = None

class ChatReq(BaseModel):
    query: str = Field(..., description="用户法律咨询问题")
    history: list[Turn] = Field(default_factory=list,
                               description="历史轮次 (含 trace 时会用于槽位继承)")
    as_of: str | None = Field(None, description="可选: 时效判定的评估时点 YYYY-MM-DD")

class CaseReq(BaseModel):
    case_no: str
    claimed_cause: str | None = None

class TemporalReq(BaseModel):
    law_short: str
    article_no: int | None = None
    as_of: str | None = None

@app.get("/health")
def health() -> dict:
    s = get_settings()
    d = {
        "status": "ok", "uptime_s": round(time.time() - _STARTED, 1),
        "provenance": s.provenance(), "router_loaded": _ROUTER is not None,
        "load_error": _LOAD_ERROR,
    }

```

```
}
# 回答模型与草稿来源的**当前生效值**（不联网、秒级）。
# 为什么单列一块：换模型/换草稿源之后，“这次服务到底是谁在回答”必须一眼可查，
# 而不是靠翻配置推（历史上出现过口径漂移，见 docs/deviations.md D21/D29/D30）。
d["llm"] = {
    "provider": s.llm_provider,
    "backend": s.llm_backend,
    "model": s.model_label("causal"),
    "api_base": s.deepseek_base_url if s.llm_provider == "deepseek" else None,
    "thinking": bool(s.deepseek_thinking) if s.llm_provider == "deepseek" else None,
    "key_present": bool(s.deepseek_api_key),
    "draft_source": s.draft_source,
    "draft_model": s.model_label("draft"),
    "note": ("DeepSeek API 调用需要网络；本字段只反映配置，不发起请求。"
            "连通性用 scripts/check_deepseek.py 验。"),
}
try:
    from lawgate.cache import get_cache

    d["cache"] = get_cache().stats()
except Exception: # noqa: BLE001
    pass
return d

@app.post("/chat")
def chat(req: ChatReq) -> dict:
    r = get_router()
    meta: dict = {}
    if req.as_of:
        meta["temporal"] = {"as_of": req.as_of}
    hist = [t.model_dump() for t in req.history]
    out = r.answer(req.query, hist, meta=meta)
    return {"answer": out.get("answer", ""), "trace": out.get("trace", {}),
            "provision": out.get("provision")}

def _sse_frame(name: str, data: dict) -> str:
    """一帧 SSE。

    ``ensure_ascii=False`` + 显式 charset 是硬要求：中文若被转成 ``\\uXXXX``
    倒还能看，但如果响应头不带 charset 而客户端按 ISO-8859-1 解码，
    中文就会在**浏览器侧**变成乱码（本项目在上一项目上踩过，见 D 系列教训）。
    ``json.dumps`` 本身保证不会有裸换行破坏帧边界。
    """
    return f"event: {name}\ndata: {_json.dumps(data, ensure_ascii=False)}\n\n"

@app.post("/chat/stream")
def chat_stream(req: ChatReq) -> StreamingResponse:
```

```

"""流式问答 (SSE)。事件顺序: stage* → delta* → done[→ error]。"""
r = get_router()          # 惰性加载在此发生 (同步端点 → 在线程池里, 不卡事件循环)
meta: dict = {}
if req.as_of:
    meta["temporal"] = {"as_of": req.as_of}
hist = [t.model_dump() for t in req.history]

def gen():
    try:
        for ev in r.answer_stream(req.query, hist, meta=meta):
            kind = ev.get("event", "delta")
            if kind == "delta":
                yield _sse_frame("delta", {"text": ev.get("text", "")})
            elif kind == "stage":
                yield _sse_frame("stage",
                                   {k: v for k, v in ev.items() if k != "event"})
            elif kind == "done":
                yield _sse_frame("done", {
                    "answer": ev.get("answer", ""),
                    "trace": ev.get("trace", {}),
                    "provision": ev.get("provision"),
                })
    except Exception as exc: # noqa: BLE001 — 异常也要成帧, 否则前端只看到"连接断了"
        yield _sse_frame("error", {"message": f"{type(exc).__name__}: {exc}"})

return StreamingResponse(
    gen(),
    media_type="text/event-stream; charset=utf-8",
    headers={"Cache-Control": "no-cache", "Connection": "keep-alive",
             "X-Accel-Buffering": "no"},
)

@app.post("/verify_case")
def verify_case(req: CaseReq) -> dict:
    from lawgate.channel.b_case_verify import CaseNoVerifier
    from lawgate.gate.intent import extract_case_no

    parsed = extract_case_no(req.case_no)
    if parsed is None and req.case_no:
        parsed = {"normalized": req.case_no.strip()[:60]}
    v = CaseNoVerifier().verify(parsed, req.claimed_cause)
    return v.to_dict()

@app.post("/temporal")
def temporal(req: TemporalReq) -> dict:
    from datetime import date

    from lawgate.channel.b_temporal import TemporalChecker

```

```

as_of = None
if req.as_of:
    as_of = date.fromisoformat(req.as_of)
tc = TemporalChecker()
if req.article_no:
    v = tc.check_provision(req.law_short, req.article_no, as_of)
else:
    v = tc.check_law(req.law_short, as_of)
d = v.to_dict()
d["replacement_map"] = tc.replacement_map(req.law_short, req.article_no)
return d

@app.get("/trace/schema")
def trace_schema() -> dict:
    return {
        "channel": "B=确定性结构化 / A=直接生成 / C=语义检索生成",
        "u": "门控决策分 ([0,1], 越大越倾向检索)",
        "u_signal": "草稿 logprob 的不确定性信号原始值",
        "u_complexity": "确定性复杂度评分",
        "draft_source": "本次 u 的草稿来自谁: api (回答模型的 API logprobs) / "
            "local (本机小模型) / rule (确定性伪分布) / "
            "channel_b (通道 B 未取草稿)",
        "draft_attempts": "换源过程: 每一次尝试的来源、耗时、成功与否及失败原因",
        "draft_seconds": "取草稿耗时 (秒)",
        "gate_source": "neural_signal | complexity —— 本桶实际采用的闸门",
        "router_mode": "hybrid | signal | complexity",
        "tau_b": "该桶的校准阈值 (dev 集校准)",
        "bucket": "b1概念/b2法条/b3案例/b4多轮",
        "decision": "人类可读的判定串",
        "slots": "抽取到的槽位 (含继承结果)",
        "slots_inherited": "实际继承的槽位名",
        "validity_status": "时效状态 (现行有效/已修订/已废止/尚未生效/查无此条)",
        "case_verify": "案号核验四级判定结果",
        "source_url": "条文官方来源",
        "latency_ms": "端到端延迟",
        "n_retrieval_calls": "本次检索调用次数 (0/1)",
        "retrieval": "检索命中明细 (doc_id/score/type/law/article)",
    }

def main() -> int:
    import argparse

    import uvicorn

    ap = argparse.ArgumentParser()
    ap.add_argument("--host", default="127.0.0.1")
    ap.add_argument("--port", type=int, default=8000)

```

```
args = ap.parse_args()
uvicorn.run("lawgate.api.app", host=args.host, port=args.port,
            log_level="info")

return 0

if __name__ == "__main__":
    raise SystemExit(main())

# =====
# 文件: lawgate/api/ui.py
# =====
# -*- coding: utf-8 -*-
"""演示系统·Gradio 双栏对照界面（手册 S7.2 / S7.3）。

左栏：无门控问答（对照组，模拟“通用模型直接回答法律问题”）
右栏：CA-LegalGate 三通道路由结果
下方：Trace 面板（通道 / u / τ_b / 桶 / 槽位继承 / 时效状态 / 案号核验 / 来源 / 延迟）

启动：
python -m lawgate.api.ui --port 7860

演示话术（手册 S7.3 的 4 步，界面里做成了 4 个预设按钮）：
1. 《合同法》第52条 → 左栏照引废止法；右栏出废止警示 + 民法典替代条 + 沿革链
2. 编造案号 → 右栏“不存在，疑似编造”
3. 真实案号 + 错误案由 → 右栏“存在但案由不符”
4. 看 Trace 面板字段逐项亮起

**流式输出**：两栏都是边生成边显示（见 ``respond_stream``），且**两栏生成长度上限相同**
（``gen_max_tokens``），默认 192 token，见 ``docs/deviations.md` D28-3）。
延迟取决于后端：走 DeepSeek API 单条 1-3 s；走本机权重（**现行默认
models/Qwen3-4B**，CPU fp16；更早是 fuzi-mingcha 6.7B, D29）则慢得多，
那种情况下不流式就是长时间白屏。
"""

from __future__ import annotations

import argparse
import html
import json
import os
import queue
import sys
import threading
import time

os.environ.setdefault("KMP_DUPLICATE_LIB_OK", "TRUE")
os.environ.setdefault("TOKENIZERS_PARALLELISM", "false")

import gradio as gr # noqa: E402
```

```

from lawgate.config import get_settings # noqa: E402

STATE: dict = {"router": None, "ctx": None, "base_llm": None, "error": None,
               "max_tokens": None, "notes": []}

def gen_max_tokens() -> int:
    """两栏统一的**最大生成长度**（唯一来源）。

    为什么必须统一：这是“双栏对照”演示，左栏是无门控直答、右栏是律核门控路径，
    两者若上限不同，长答案看起来“更完整”，对照本身就不公平了。
    取值顺序：`LAWGATE_UI_MAXTOK`（由 scripts/serve_one.py 写进 Settings）→
    `configs/base.yaml: max_new_tokens`（默认见该文件注释）。
    """
    return int(get_settings().max_new_tokens)

def ensure_loaded():
    """惰性加载：首次交互时才加载模型与向量库。

    关键：把生成长度**在构造 router 时就定死**（`RouterOptions.max_new_tokens`）。
    右栏（通道 A/C）的 `max_tokens` 取自 router options（见 `lawgate/router.py`），
    不在这里设就会退回 dataclass 默认值 192 —— 与左栏的 `Settings.max_new_tokens`
    不一致，于是两栏“问题不同长”。评测侧也是同一套约定，见
    `lawgate/eval/run_exp.py`（“LegalGate 的生成参数走 router options，
    必须与其它基线一致，否则对比不公平”）。
    """
    if STATE["router"] is None and STATE["error"] is None:
        try:
            from lawgate.eval.run_exp import build_context
            from lawgate.router import RouterOptions

            mt = gen_max_tokens()
            ctx = build_context(options=RouterOptions(max_new_tokens=mt))
            STATE["ctx"] = ctx
            STATE["router"] = ctx.router
            STATE["base_llm"] = ctx.llm
            STATE["max_tokens"] = mt
        except Exception as exc: # noqa: BLE001
            STATE["error"] = f"{type(exc).__name__}: {exc}"
    return STATE["router"]

def column_max_tokens(router) -> int:
    """本次提问两栏实际使用的生成上限。

    以 **router options 为准**（右栏的真实取值），左栏显式传同一个数——
    这样“两栏同长”是由构造保证的，而不是靠两处配置碰巧相等。
    """

```

```
opt = getattr(router, "opt", None)
mt = getattr(opt, "max_new_tokens", None)
return int(mt or gen_max_tokens())

def real_case_no() -> str:
    """从案号库里取一个真实（合成库内）案号，供预设按钮使用。"""
    import sqlite3

    s = get_settings()
    conn = sqlite3.connect(s.db_path)
    try:
        r = conn.execute(
            "SELECT case_no, cause_action FROM case_registry "
            "WHERE cause_action='民间借贷纠纷' LIMIT 1").fetchone()
    finally:
        conn.close()
    return r[0] if r else "(2022)沪01民终12345号"

PRESETS = {
    "① 已废止法律（杀手用例）": "《合同法》第52条规定哪些情形合同无效？",
    "② 编造案号": "(2099)沪01民终88888号 这个案子是什么案由？",
    "③ 真实案号 + 错误案由": None, # 运行时填入
    "④ 法律是否仍有效": "担保法现在还有用吗？",
    "⑤ 纯概念题（应走直答/检索，不进 B）": "什么是离婚冷静期？",
    "⑥ 多轮追问（槽位继承）": "那我能主张多少利息？",
}

def build_trace_html(trace: dict) -> str:
    if not trace:
        return "<i>无 trace</i>"

    def esc(x) -> str:
        return html.escape(str(x))

    temporal = trace.get("temporal") or {}
    cv = trace.get("case_verify") or {}
    rows = [
        ("通道", trace.get("channel")),
        ("门控来源", trace.get("gate_source")),
        ("决策", trace.get("decision")),
        ("u（决策分）", trace.get("u")),
        ("u_signal（神经信号）", trace.get("u_signal")),
        ("u_complexity（复杂度）", trace.get("u_complexity")),
        # 草稿来源必须显示（D30）：回答模型换成 API 后，u 可能来自 API 自己的
        # logprobs、本机小模型或确定性伪分布，三者含义不同，混在一起看就是错的
        ("草稿来源", trace.get("draft_source")),
        ("草稿耗时(ms)", (round(float(trace["draft_seconds"]) * 1000))
    ]
```

```

        if trace.get("draft_seconds") is not None else None)),
        ("τb", trace.get("tau_b")),
        ("桶", trace.get("bucket")),
        ("槽位", json.dumps(trace.get("slots") or {}, ensure_ascii=False)),
        ("继承槽位", trace.get("slots_inherited")),
        ("时效状态", temporal.get("status") or trace.get("validity_status")),
        ("案号核验", cv.get("level")),
        ("来源", trace.get("source_url")),
        ("延迟(ms)", trace.get("latency_ms")),
        ("检索调用", trace.get("n_retrieval_calls")),
    ]
    body = "".join(
        f"<tr><td style='padding:2px 8px;color:#666;white-space:nowrap'>{esc(k)}</td>"
        f"<td style='padding:2px 8px'><b>{esc(v)}</b></td></tr>"
        for k, v in rows)
    warn = ""
    if trace.get("must_show_warning"):
        warn = ("<div style='background:#fff4e5;border-left:4px solid #e8a33d;"
                "padding:6px 10px;margin-bottom:8px'>    本条命中时效/核验风险, "
                "必须向用户显式提示</div>")
    retr = trace.get("retrieval") or {}
    hits = retr.get("hits") or []
    hit_html = ""
    if hits:
        items = "".join(
            f"<li>score={h.get('score')} type={esc(h.get('type'))} "
            f"law={esc(h.get('law'))} article={esc(h.get('article'))} "
            f"case={esc(h.get('case_no'))}</li>" for h in hits[:8])
        hit_html = f"<details><summary>检索命中 {len(hits)} 条</summary>" \
            f"<ul style='font-size:12px'>{items}</ul></details>"
    steps = trace.get("steps") or []
    return (f"<table style='font-size:13px;line-height:1.5'>{body}</table>"
            f"<div style='font-size:12px;color:#555;margin-top:6px'>"
            f"通道 B 步骤: {esc(steps)}</div>{hit_html}")

def _stage_html(ev: dict) -> str:
    """stage 事件的“正在做什么”提示（流式期间 trace 还没出来，先给进展）。"""
    stage = ev.get("stage")
    if stage == "channel_b":
        return "<i>    通道 B（确定性结构化）：查法条/案号库中...</i>"
    if stage == "retrieval":
        return ("<i>    通道 C：检索法条与案例中..."
                f" (u={ev.get('u')} &gt; τ={ev.get('tau_b')}) </i>")
    if stage == "generate":
        return (f"<i>    通道 {html.escape(str(ev.get('channel')))}：生成中..."
                f" (桶 {html.escape(str(ev.get('bucket')))}, "
                f"u={ev.get('u')} / τ={ev.get('tau_b')}) </i>")
    return "<i>路由中...</i>"

```

```
def respond_stream(query: str, history: list, as_of: str):
    """流式版应答 (**生成器**): yield `` (左栏, 右栏, trace_html, state, 备注)``。

    Gradio 的流式输出 = 处理函数是生成器, 每次 yield 刷新一次组件。因此旧版
    "算完一次性 return" 的 ``respond`` 已由本函数取代 (同签名、同输出位序)。

    两栏为什么必须并行:
    左栏 (无门控直答) 与右栏 (律核) 各要十几秒 CPU 生成。若串行, "先左后右"
    或"先右后左"都会让另一栏一直空白。故左栏丢后台线程、分片写进 ``queue``,
    本线程在推进右栏事件的间隙顺手取走——两栏同时长出来。
    """
    history = history or []
    router = ensure_loaded()
    if router is None:
        err = f"**加载失败**: {STATE['error']}"
        yield err, err, "<i>trace 不可用</i>", history, ""
        return
    if not query.strip():
        yield "", "", "<i>请输入问题</i>", history, ""
        return

    meta = {}
    if as_of.strip():
        meta["temporal"] = {"as_of": as_of.strip()}

    # 两栏共用的生成上限 (右栏取 router options, 左栏显式传同一个数)
    mt = column_max_tokens(router)

    # ----- 左栏: 后台线程
    q: queue.Queue = queue.Queue()

    def _left_worker() -> None:
        try:
            # max_tokens 必须显式给: 不传就会用 HFLLM 自己的默认值 (Settings 值),
            # 与右栏的 router options 是两个来源, 容易再次漂移。
            for piece in STATE["base_llm"].generate_stream(query, history,
                                                             context=None,
                                                             max_tokens=mt):
                if piece:
                    q.put(("delta", piece))
                q.put(("end", ""))
            except Exception as exc: # noqa: BLE001
                q.put(("error", f"{type(exc).__name__}: {exc}"))

    left_buf = ""
    left_err = ""
    left_done = False

    def _take_left(kind: str, payload: str) -> None:
```

```

nonlocal left_buf, left_err, left_done
if kind == "delta":
    left_buf += payload
elif kind == "error":
    left_err = f"[左栏生成失败] {payload}"
    left_done = True
else:
    left_done = True

def _drain_left() -> None:
    while True:
        try:
            _take_left(*q.get_nowait())
        except queue.Empty:
            return

threading.Thread(target=_left_worker, daemon=True,
                  name="lawgate-ui-left").start()

# ----- 右栏: 本线程
right_buf = ""
right_err = ""
provision_tail = ""
trace: dict = {}
status_html = "<i>路由中...</i>"
t0 = time.time()
last_flush = 0.0

def _snapshot(final: bool = False) -> tuple:
    left = (left_err or left_buf).strip()
    right = (right_err or (right_buf + provision_tail)).strip()
    if trace:
        trace_html = build_trace_html(trace)
    elif right_err:
        trace_html = f"<i>{html.escape(right_err)}</i>"
    else:
        trace_html = status_html
    ch = trace.get("channel")
    note = (f"左栏=无门控直答 右栏=律核 ({ch or '路由中'}) "
           f"两栏生成长度上限均 {mt} token "
           f"{'已完成' if final else '流式输出中...'}")
    if final:
        note += f" 右栏耗时 {(time.time() - t0) * 1000:.0f} ms"
    return left, right, trace_html, note

try:
    for ev in router.answer_stream(query, history, meta=meta):
        _drain_left()
        kind = ev.get("event")
        if kind == "stage":

```

```

        status_html = _stage_html(ev)
    elif kind == "delta":
        right_buf += ev.get("text", "")
    elif kind == "done":
        # done.answer 是权威全文：用它覆盖分片拼接，避免首尾空白/特殊 token
        # 造成的细微差异（分片只是过程，最终口径以它为准）。
        right_buf = ev.get("answer") or right_buf
        trace = ev.get("trace") or {}
        if ev.get("provision"):
            provision_tail = "\n\n【结构化命中】" + "; ".join(
                f"《{p.get('law_short')}》{p.get('article_label')}"
                for p in ev["provision"][:5])
        now = time.time()
        # 节流：token 级事件在 CPU 上约 3-8 个/秒，但缓存命中时会一次涌出很多分片，
        # 不节流会把 WebSocket 打爆（每次 yield 都是一次 Markdown 重渲染）。
        if now - last_flush >= 0.06 or kind in ("done", "stage"):
            last_flush = now
            left, right, trace_html, note = _snapshot()
            yield left, right, trace_html, history, note
except Exception as exc: # noqa: BLE001
    right_err = f"[右栏失败] {type(exc).__name__}: {exc}"

# 右栏收工了，但左栏可能还在生成（两栏各十几秒，谁先完不一定）：
# 继续把左栏的尾巴取干净，否则界面会停在半句话上。
while not left_done:
    try:
        _take_left(*q.get(timeout=0.2))
    except queue.Empty:
        continue
    now = time.time()
    if now - last_flush >= 0.06:
        last_flush = now
        left, right, trace_html, note = _snapshot()
        yield left, right, trace_html, history, note

_drain_left()
left, right, trace_html, note = _snapshot(final=True)
history = history + [{"query": query, "answer": right, "trace": trace}]
yield left, right, trace_html, history, note

```

```

def build_demo() -> gr.Blocks:
    rc = real_case_no()
    PRESETS["③ 真实案号 + 错误案由"] = f"{rc} 这个案子是劳动争议，对吗？"

    s = get_settings()
    prov = s.provenance()
    api_mode = prov.get("llm_provider") == "deepseek"
    model_name = prov["causal_model"]
    if api_mode:

```

```
wait_line = ("每次提问约 **1-3 秒**出答案（DeepSeek API，关闭思考模式）；"  
            "命中内容缓存后 ≈ 0.05 s。")  
model_line = (f"**{model_name}**（DeepSeek 官方 API：{prov['llm_api_base']}）；"  
             f"思考模式 {'开' if prov.get('llm_thinking') else '关'}）")  
head_note = ("语言模型走 DeepSeek 官方 API，**不需要**在本机加载权重。"  
            f"门控草稿用本机 {prov.get('draft_model')}：**首次提问**会多花时间"  
            "把它加载进内存，之后每次提问都是 1-3 秒。")  
else:  
    wait_line = ("CPU 上首字几秒到几十秒出现，整段受生成长度上限约束"  
                "（命中缓存 ≈ 0.1 s）。实测数字见 `docs/deviations.md` D38。")  
    model_line = (f"**{model_name}**（{prov.get('causal_model_dtype', 'auto')} 精度，"  
                 "本机权重）")  
    head_note = ("语言模型在本机 CPU 上运行。门控草稿与回答模型是**同一份权重**"  
                "（手册 S3.5 原设计：草稿与正式生成共享模型与前缀，prefill 不翻倍），"  
                "因此没有`额外再加载一个草稿模型`这笔开销。")  
  
with gr.Blocks(title="CA-LegalGate 律核 • 双栏对照") as demo:  
    gr.Markdown(  
        "## 律核 LawGate • 双栏对照演示\n"  
        "**左栏**：无门控直答（通用模型直接回答，作为对照组） | "  
        "**右栏**：CA-LegalGate 三通道路由（B 确定性结构化 / A 直接生成 / C 语义检索）\n\n"  
        "> 说明手册 S7.3 演示话术：点下面的预设按钮，重点看右栏的**废止警示**、"  
        "**案号核验**与下方 **Trace 面板**。\n\n"  
        f"> 两栏都是**流式输出**（边生成边显示），且**生成长度上限相同"  
        f"（{gen_max_tokens()} token）**：{wait_line}\n\n"  
        f"> {head_note}")  
  
    with gr.Row():  
        with gr.Column(scale=1):  
            gr.Markdown("### 左栏：无门控直答（对照）")  
            left = gr.Markdown()  
        with gr.Column(scale=1):  
            gr.Markdown("### 右栏：律核（可解释路由）")  
            right = gr.Markdown()  
  
    with gr.Row():  
        inp = gr.Textbox(label="你的法律咨询", lines=3,  
                        placeholder="例如：《合同法》第52条规定哪些情形合同无效？")  
  
    with gr.Row():  
        as_of = gr.Textbox(label="评估时点（可选，YYYY-MM-DD；留空=今天），"  
                          value="", scale=1)  
        btn = gr.Button("提问", variant="primary", scale=1)  
  
    gr.Markdown("#### 演示预设")  
    with gr.Row():  
        for label, q in PRESETS.items():  
            if q is None:  
                continue  
            gr.Button(label, size="sm").click(  
                lambda _q: _q, outputs=inp, queue=False)
```

```

gr.Markdown("### Trace 面板（每一步决策可解释、可复核、有官方来源）")
trace_html = gr.HTML()
note = gr.Markdown()
state = gr.State([])

# 流式: respond_stream 是生成器, Gradio 会把每次 yield 推给浏览器（需要队列,
# Blocks 默认已开启; 此处显式声明, 避免以后有人误关）。
demo.queue()
btn.click(respond_stream, [inp, state, as_of],
          [left, right, trace_html, state, note])
inp.submit(respond_stream, [inp, state, as_of],
          [left, right, trace_html, state, note])

# 页脚的口径必须与**实际生效**的模型一致: 早期写死的 0.5B 早已过时
# (D21), 后来换成 fuzi-mingcha (D29), 再后来默认走 DeepSeek API (D30),
# 2026-09-13 起默认又回到本机权重 models/Qwen3-4B (D38) ——
# 故这里一律从 Settings.provenance() 动态取, 避免再次写错。
draft_line = (
    f"门控草稿来源 `{prov.get('draft_source')}`"
    f"(本机 {prov.get('draft_model')} → API logprobs → 确定性评分;"
    "回答模型走本地时草稿直接复用回答模型本身, 手册 S3.5;"
    "每条 trace 的 **草稿来源** 行会写明本次实际用了哪条,"
    "见 `docs/deviations.md` D30 / D38)。")
gr.Markdown(
    "---\n"
    f"**说明**: 本项目运行在 {prov['hardware']} 环境, 语言模型为 "
    f"{model_line}, "
    f"单栏最多生成 {gen_max_tokens()} token, 向量模型为 {prov['embed_model']}; "
    "法条语料为人工录入的关键条文"
    "(PENDING_FLK_VERIFICATION), 案号库为合成数据。 \n\n"
    f"{draft_line}\n\n"
    "详见 `docs/DATA_GAP.md` 与 `docs/deviations.md`。")
return demo

def main() -> int:
    ap = argparse.ArgumentParser()
    ap.add_argument("--host", default="127.0.0.1")
    ap.add_argument("--port", type=int, default=7860)
    ap.add_argument("--share", action="store_true")
    args = ap.parse_args()
    build_demo().launch(server_name=args.host, server_port=args.port,
                        share=args.share, show_error=True)
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

```
# =====
# 文件: lawgate/cache.py
# =====
# -*- coding: utf-8 -*-
"""生成/草稿结果缓存 (E1 的算力前提)。
```

为什么必须有:

E1 要在 6 个方法 $\times$ 3 个 seed $\times$ 864 条测试题上跑生成。本机 CPU 单条 128 token 生成约 10-15 秒, 总需求超过 40 机时。但**贪心解码是确定性的**, 同一 (模型, prompt, max_tokens) 必然得到同一结果, 且不同方法之间存在大量重复:

NeverRAG / ComplexityRouter(直答) / TARG(直答) / LegalGate(通道A) $\rightarrow$ 同一 prompt
 AlwaysRAG / ComplexityRouter(检索) / TARG(检索) / LegalGate(通道C) $\rightarrow$ 同一 prompt

缓存把这些重复折叠掉, 实测把唯一生成数从 ~1.5 万降到 ~1.7 千。

缓存是**内容寻址**的 (sha256 of model|kind|max_tokens|prompt), 因此:

- * 换模型自动失效;
- * 结果可复现、可审计;
- * 缓存文件本身可作为“没有偷跑”的证据 (记录每条 prompt 的哈希)。

```
"""
```

```
from __future__ import annotations
```

```
import hashlib
import json
import sqlite3
import threading
from pathlib import Path
```

```
SCHEMA = """
```

```
CREATE TABLE IF NOT EXISTS gen_cache (
    key          TEXT PRIMARY KEY,
    kind         TEXT NOT NULL,
    model        TEXT NOT NULL,
    max_tokens   INTEGER,
    payload      TEXT NOT NULL,
    n_tokens     INTEGER,
    seconds      REAL,
    created      TEXT DEFAULT (datetime('now'))
);
CREATE INDEX IF NOT EXISTS idx_gc_kind ON gen_cache(kind, model);
CREATE TABLE IF NOT EXISTS gen_stats (
    key TEXT PRIMARY KEY,
    hits INTEGER DEFAULT 0,
    misses INTEGER DEFAULT 0
);
"""
```

```
def make_key(model: str, kind: str, max_tokens: int | None, prompt: str) -> str:
    h = hashlib.sha256()
    h.update(f"{model}\x00{kind}\x00{max_tokens}\x00".encode("utf-8"))
    h.update(prompt.encode("utf-8"))
    return h.hexdigest()
```

```
class GenCache:
    """SQLite 支持的线程/进程安全缓存。"""

    def __init__(self, path: str | Path = "data/kb/gen_cache.db",
                 enabled: bool = True):
        self.path = Path(path)
        self.enabled = enabled
        self._lock = threading.Lock()
        self._local = threading.local()
        if self.enabled:
            self.path.parent.mkdir(parents=True, exist_ok=True)
            with self._conn() as c:
                c.executescript(SCHEMA)

    def _conn(self) -> sqlite3.Connection:
        conn = getattr(self._local, "conn", None)
        if conn is None:
            conn = sqlite3.connect(str(self.path), timeout=60.0,
                                   check_same_thread=False)
            conn.execute("PRAGMA journal_mode=WAL")
            self._local.conn = conn
        return conn

    def get(self, key: str):
        if not self.enabled:
            return None
        with self._lock:
            row = self._conn().execute(
                "SELECT payload FROM gen_cache WHERE key=?", (key,)).fetchone()
            if row is None:
                return None
            return json.loads(row[0])

    def put(self, key: str, kind: str, model: str, max_tokens: int | None,
            payload, n_tokens: int = 0, seconds: float = 0.0) -> None:
        if not self.enabled:
            return
        with self._lock:
            conn = self._conn()
            conn.execute(
                "INSERT OR REPLACE INTO gen_cache"
                "(key, kind, model, max_tokens, payload, n_tokens, seconds) "
                "VALUES (?, ?, ?, ?, ?, ?, ?)",
```

```

        (key, kind, model, max_tokens,
         json.dumps(payload, ensure_ascii=False), n_tokens, seconds))
    conn.commit()

def hit_miss(self, key: str, hit: bool) -> None:
    if not self.enabled:
        return
    with self._lock:
        conn = self._conn()
        conn.execute(
            "INSERT INTO gen_stats(key,hits,misses) VALUES (?, ?, ?) "
            "ON CONFLICT(key) DO UPDATE SET "
            "hits=hits+excluded.hits, misses=misses+excluded.misses",
            (key, 1 if hit else 0, 0 if hit else 1))
        conn.commit()

def stats(self) -> dict:
    if not self.enabled:
        return {"enabled": False}
    with self._lock:
        rows = self._conn().execute(
            "SELECT kind, COUNT(*) n, SUM(n_tokens) tok, SUM(seconds) sec "
            "FROM gen_cache GROUP BY kind").fetchall()
        tot = self._conn().execute(
            "SELECT COUNT(*) n, COALESCE(SUM(n_tokens),0) t, "
            "COALESCE(SUM(seconds),0) s FROM gen_cache").fetchone()
    return {
        "enabled": True,
        "by_kind": {k: {"entries": n, "tokens": t or 0, "seconds": round(s or 0, 1)}
                     for k, n, t, s in rows},
        "total_entries": tot[0], "total_tokens": tot[1],
        "total_compute_seconds": round(tot[2], 1),
    }

def clear(self) -> None:
    with self._lock:
        conn = self._conn()
        conn.execute("DELETE FROM gen_cache")
        conn.execute("DELETE FROM gen_stats")
        conn.commit()

```

_GLOBAL: GenCache | None = None

```

def get_cache(path: str | Path | None = None, enabled: bool = True) -> GenCache:
    global _GLOBAL
    if _GLOBAL is None or path is not None:
        _GLOBAL = GenCache(path or "data/kb/gen_cache.db", enabled=enabled)
    return _GLOBAL

```

```
# =====
# 文件: lawgate/channel/__init__.py
# =====
# -*- coding: utf-8 -*-
"""通道层: B 结构化确定通道 / A 直接生成 / C 语义检索生成。"""
```

```
# =====
# 文件: lawgate/channel/b_case_verify.py
# =====
# -*- coding: utf-8 -*-
"""通道 B • 案号三级核验 (手册 S3.3)。
```

在手册实现上的补强 (docs/deviations.md D15):

- D15.1 手册的顺序是“格式→存在→类型→案由”，但“格式非法”要靠 ``parsed`` 为空来判定；而 intent 层对“看起来像案号但代字非法”的输入会给出 ``parsed={}``，两者需区分。本实现显式区分 **格式非法** 与 **格式合法但不存在**，并把解析细节（年份/法院代字/类型代字/序号）写进 detail 便于审计。
- D15.2 增加 ``year\_range`` 合理性检查（1990—当前年+1），拦截“（1899）京01民终1号”这类格式合法但不可能的案号。
- D15.3 增加 ``claimed\_cause`` 的模糊匹配：真实案由常写作“民间借贷纠纷”，提问可能写“借贷”，故用包含关系双向匹配而非严格相等。

```
from __future__ import annotations
```

```
import sqlite3
import threading
from dataclasses import dataclass, field
from datetime import date
```

```
from lawgate.config import get_settings
```

```
LEVELS = ["格式非法", "不存在", "存在但类型不符", "存在但案由不符", "核验通过"]
```

```
@dataclass
```

```
class VerifyResult:
```

```
    level: str
    passed: bool
    detail: str
    real_case: dict | None = None
    parsed_detail: dict = field(default_factory=dict)
```

```
    def to_dict(self) -> dict:
```

```
        return {"level": self.level, "passed": self.passed, "detail": self.detail,
                "real_case": self.real_case, "parsed": self.parsed_detail}
```

```
def _cause_match(claimed: str, real: str) -> bool:
    """D15.3: 双向包含匹配（'借贷' '民间借贷纠纷' 视为一致）。"""
    if not claimed or not real:
        return True
    c = claimed.replace("纠纷", "").strip()
    r = real.replace("纠纷", "").strip()
    if not c or not r:
        return True
    return c in r or r in c

class CaseNoVerifier:
    def __init__(self, db_path: str | None = None,
                 year_range: tuple[int, int] = (1990, date.today().year + 1)):
        self.db_path = db_path or get_settings().db_path
        # 连接按线程持有 (D28): 本对象被多线程共享 (FastAPI 线程池 / 评测线程池),
        # 而 sqlite3 连接默认禁止跨线程使用。
        self._local = threading.local()
        self.year_range = year_range

    @property
    def conn(self) -> sqlite3.Connection:
        """当前线程的连接 (首次访问时建立)。"""
        c = getattr(self._local, "conn", None)
        if c is None:
            c = sqlite3.connect(self.db_path)
            c.row_factory = sqlite3.Row
            self._local.conn = c
        return c

    def verify(self, parsed: dict | None, claimed_cause: str | None = None) -> VerifyResult:
        # ---- 一级: 格式 ----
        if not parsed:
            return VerifyResult(
                "格式非法", False,
                " 未能解析出合法案号格式。合法案号形如「(2022)沪01民终12345号」: "
                "年份+法院代字+案件类型代字+序号, 四要素缺一不可。",
                None, {})

        norm = parsed.get("normalized", "")
        detail = {k: parsed.get(k) for k in
                  ("year", "court_code", "case_type", "case_type_name", "seq_no")}
        if parsed.get("abnormal_marker"):
            return VerifyResult(
                "格式非法", False,
                f" 案号「{norm}」使用了非法案件类型代字 (如「测」), "
                f"不属于最高人民法院《案号标准》认可的代字。", None, detail)

        y = parsed.get("year")
        if not y or not (self.year_range[0] <= y <= self.year_range[1]):
```

```
        return VerifyResult(
            "格式非法", False,
            f" 案号「{norm}」的年份 {y} 不在合理范围 "
            f"「{self.year_range[0]}—{self.year_range[1]} 内。", None, detail)

code = str(parsed.get("court_code") or "")
if not code or not code[0].isalpha() and not ("\u4e00" <= code[0] <= "\u9fa5"):
    return VerifyResult(
        "格式非法", False,
        f" 案号「{norm}」的法院代字「{code}」非法：法院代字须以"
        f"省级行政区简称开头。", None, detail)

# ---- 二级：存在性 ----
row = self.conn.execute(
    "SELECT * FROM case_registry WHERE case_no=?", (norm,)).fetchone()
if not row or not row["exists_in_db"]:
    return VerifyResult(
        "不存在", False,
        f" 案号 {norm} 在裁判文书库中不存在，疑似编造。请勿在正式文书中引用。",
        None, detail)

real = {"case_no": row["case_no"], "court_name": row["court_name"],
        "cause_action": row["cause_action"], "case_type": row["case_type"],
        "judgment_date": row["judgment_date"],
        "data_source": row["data_source"] if "data_source" in row.keys() else None,
        "source_url": row["source_url"] if "source_url" in row.keys() else None}

# ---- 三级：类型一致性 ----
claimed_type = parsed.get("case_type_name") or ""
if claimed_type and row["case_type"] and claimed_type != row["case_type"]:
    return VerifyResult(
        "存在但类型不符", False,
        f" 案号 {norm} 真实存在，但类型为「{row['case_type']}」，"
        f"而非所称「{claimed_type}」。", real, detail)

# ---- 四级：案由一致性 ----
if claimed_cause and row["cause_action"] and \
    not _cause_match(claimed_cause, row["cause_action"]):
    return VerifyResult(
        "存在但案由不符", False,
        f" 案号 {norm} 真实存在，但真实案由为「{row['cause_action']}」，"
        f"与所述「{claimed_cause}」不符。引用将导致法律文书失实。",
        real, detail)

return VerifyResult(
    "核验通过", True,
    f" 案号 {norm} 核验通过：{row['court_name']}, "
    f"案由「{row['cause_action']}」，{row['case_type']}。",
    real, detail)
```

```
def verify_raw(self, raw: str, claimed_cause: str | None = None) -> VerifyResult:
    """从原始文本解析并核验。"""
    from lawgate.gate.intent import extract_case_no

    parsed = extract_case_no(raw or "")
    if not parsed:
        return VerifyResult("格式非法", False,
                             f" 未能解析出合法案号格式: {raw[:60]}", None, {})
    return self.verify(parsed, claimed_cause)
```

```
def log(self, result: VerifyResult, expected: str | None = None) -> None:
    try:
        self.conn.execute(
            """INSERT INTO case_verify_log
               (case_no, claimed_cause, expected, got_level, got_passed, run_date)
               VALUES (?, ?, ?, ?, date('now'))""",
            (result.real_case["case_no"] if result.real_case else "",
             None, expected, result.level, int(result.passed)))
        self.conn.commit()
    except sqlite3.Error:
        pass # 日志表缺失不应影响核验主流程
```

```
# =====
# 文件: lawgate/channel/b_structured.py
# =====
# -*- coding: utf-8 -*-
"""通道 B • 确定性结构化主逻辑 (手册 S3.4)。
```

四条路径 (比手册多一条, 见 D12):

```
P1 案号核验      slots.case_no_parsed 非空
P2 法条+时效      law_short ^ article_no
P3 法律效力询问    law_short ^ law_validity_query (「担保法现在还有用吗」)
P4 主题条文检索    topic ^ provision_seeking ^ ^article_no
```

返回 ``None`` 表示“查不到 / 槽位不全”→ 由 router 回落到门控 (手册的 B 降级路径)。

连接口径 (D28): ``conn`` 是**按线程**惰性建的属性, 不是 ``\_\_init\_\_`` 里那一条共享连接。
 sqlite3 的连接默认禁止跨线程使用, 而 FastAPI 的同步端点 (含 ``/chat/stream``) 跑在
 Starlette 的线程池里、评测的 ``workers>1`` 也是多线程——共享一条连接会直接抛
 ``ProgrammingError: SQLite objects created in a thread can only be used in that same thread``。
 """

```
from __future__ import annotations
```

```
import sqlite3
import threading
from datetime import date
```

```
from lawgate.channel.b_case_verify import CaseNoVerifier
from lawgate.channel.b_temporal import TemporalChecker, TemporalVerdict
```

```

from lawgate.config import get_settings
from lawgate.gate.intent import TOPIC_KEYWORDS_FALLBACK

class ChannelB:
    def __init__(self, db_path: str | None = None):
        self.db_path = db_path or get_settings().db_path
        self._local = threading.local()
        self.temporal = TemporalChecker(self.db_path)
        self.verifier = CaseNoVerifier(self.db_path)
        self._topic_kw = self._load_topic_keywords()
        self._causes = self._load_causes()

    @property
    def conn(self) -> sqlite3.Connection:
        """当前线程的连接（首次访问时建立）。"""
        c = getattr(self._local, "conn", None)
        if c is None:
            c = sqlite3.connect(self.db_path)
            c.row_factory = sqlite3.Row
            self._local.conn = c
        return c

# ----- 工具
def _load_causes(self) -> list[str]:
    """案由词表 (D26-2)：取自 case_registry 的 cause_action 去重值，长词优先。

    为什么不能拿 slots.topic 当“所称案由”：topic 是**主题标签**
    （民间借贷 / 劳动争议 /...），不是案由（民间借贷纠纷 / 房屋租赁合同纠纷 /...）。
    之前 ChannelB 直接把 slots.topic 传给核验器，于是
    「（2019）鲁01民终1242号 这个案子是劳动争议，对吗？」——这句话里
    “劳动争议”根本不在 topic_keyword 的关键词表里（那是 加班费/辞退/裁员/劳动合同...），
    slots.topic 为 None → 核验器拿不到 claimed_cause → 误报「核验通过」。
    UI 的演示预设③ 用的就是这句话，现场会与 README 承诺的「存在但案由不符」对不上。
    """
    try:
        rows = self.conn.execute(
            "SELECT DISTINCT cause_action FROM case_registry "
            "WHERE cause_action IS NOT NULL AND cause_action <> ''").fetchall()
    except sqlite3.Error:
        return []
    return sorted((r[0] for r in rows), key=lambda s: -len(s))

def _claimed_cause(self, query: str) -> str | None:
    """从提问里扫出用户**声称**的案由；没声称就返回 None（=不做案由比对）。"""
    for c in self._causes:
        if c in query:
            return c
    return None

```

```
def _load_topic_keywords(self) -> dict[str, list[str]]:
    d: dict[str, list[str]] = {}
    try:
        for r in self.conn.execute("SELECT topic, keyword FROM topic_keyword"):
            d.setdefault(r["topic"], []).append(r["keyword"])
    except sqlite3.Error:
        pass
    return d or {k: list(v) for k, v in TOPIC_KEYWORDS_FALLBACK.items()}
```

```
def _topic_provisions(self, topic: str) -> list:
```

"""按主题词取“现行有效”条文 (D26-3)。

原先 P4 走 ``provision\_fts MATCH``，但建库时用的是 ``tokenize='unicode61'``，而 unicode61 **不切分中文**——它把一整串连续汉字当成 **一个** token。实测：

```
MATCH '利息'          → 0 行
MATCH '"支付利息"'    → 0 行
MATCH '民法典'        → 96 行    (law_short 列里“民法典”恰好是独立 token)
```

而 ``\_fts\_query('民间借贷')`` 生成的是一长串中文 OR 条件，于是**永远 0 行**，``\_topic\_fts\_miss->C`` 后被 ``return None`` 吃掉 —— P4 自建成起从未命中过一次。

这里改用 ``provision\_keywords`` 做**等值**匹配（该表是建库时按条文标注的主题词，没有分词问题），再回连 ``legal\_provisions``。同一 (law_short, article_no) 可能有多“款/项”行：按 item_no 排序后只取首行，避免同一张条文重复渲染。

"""

```
kws = [k for k in (self._topic_kw.get(topic) or [topic]) if k]
if not kws:
    return []
ph = ", ".join("? " * len(kws))
try:
    rows = self.conn.execute(
        f"""SELECT p.law_short, p.article_label, p.article_no, p.item_no,
               p.text, p.source_url, p.validity_status, k.weight
        FROM provision_keywords k
        JOIN legal_provisions p
          ON p.law_short = k.law_short AND p.article_no = k.article_no
        WHERE k.keyword IN ({ph})
          AND p.validity_status = '现行有效'
        ORDER BY k.weight DESC, p.law_short, p.article_no, p.item_no""",
        tuple(kws)).fetchall()
except sqlite3.Error:
    return []
picked, seen = [], set()
for r in rows:
    key = (r["law_short"], r["article_no"])
    if key in seen:
        continue
    seen.add(key)
    picked.append(r)
```

```

        if len(picked) >= 8:
            break
    return picked

@staticmethod
def _render(rows, max_rows: int = 8) -> str:
    out = []
    for r in rows[:max_rows]:
        head = f"《{r['law_short']}》 {r['article_label']}"
        if r["item_no"]:
            head += f" {r['item_no']}"
        body = f"{head} {r['text']}"
        url = r["source_url"] if "source_url" in r.keys() else ""
        body += f"\n(来源: {url})" if url else ""
        out.append(body)
    return "\n\n".join(out)

def _compose_warning(self, tv: TemporalVerdict, rep: dict | None) -> str:
    parts = [tv.warning or ""]
    if rep:
        prov = self.temporal.fetch_current(rep["law_short"], rep["article_no"]) \
            if rep.get("article_no") else []
        if prov:
            parts.append(f"【现行规定】《{rep['law_short']}》"
                        f"{prov[0]['article_label']}: {prov[0]['text']}")
            if prov[0]["source_url"]:
                parts.append(f"(来源: {prov[0]['source_url']})")
        else:
            parts.append(f"【现行规定】《{rep.get('law_short', '')}》: "
                        f"{rep.get('note', '')}")
        if rep.get("note"):
            parts.append(f"【替代说明】{rep['note']}")
    if tv.lifecycle_chain and len(tv.lifecycle_chain) > 1:
        parts.append(f"【法律沿革】{' → '.join(tv.lifecycle_chain)}")
    return "\n\n".join(p for p in parts if p)

# ----- 主逻辑 -----
def run(self, slots, raw_query: str = "", as_of: date | None = None) -> dict | None:
    trace: dict = {"channel": "B", "slots": slots.to_dict(), "steps": []}
    as_of = as_of or date.today()

    # ----- P1 案号核验 -----
    if slots.case_no_parsed or slots.case_no_raw:
        # D26-2: 所称案由优先从提问里直接扫出来 (案由词表来自 case_registry),
        # slots.topic 只作为兜底——它只是主题标签, 粒度不等于案由。
        claimed = self._claimed_cause(raw_query) or slots.topic
        v = self.verifier.verify(slots.case_no_parsed, claimed)
        trace["case_verify"] = v.to_dict()
        trace["steps"].append("case_no_verify")
        trace["validity_status"] = None

```

```

        trace["source_url"] = (v.real_case or {}).get("source_url") or None
        ans = v.detail
        if v.passed and v.real_case:
            ans += (f"\n\n【文书信息】{v.real_case['court_name']} | "
                    f"案由: {v.real_case['cause_action']} | "
                    f"裁判日期: {v.real_case['judgment_date']}")
            ds = v.real_case.get("data_source")
            if ds == "SYNTHETIC":
                ans += ("\n\n    【数据溯源】本案号来自**合成案号库**"
                        " (data_source=SYNTHETIC), 用于系统联调, "
                        "不对应真实案件, 不得作为真实引用依据。")
            else:
                src_url = v.real_case.get("source_url")
                ans += ("\n\n    【数据溯源】本案号来自**真实裁判文书库**"
                        f" (data_source={ds}) "
                        + (f", 来源: {src_url}" if src_url else "")
                        + "。引用请以上述案号在中国裁判文书网核验为准。")
        return {"answer": ans, "trace": trace,
                "must_show_warning": not v.passed}

# ----- P2 法条 + 时效 -----
if slots.law_short and slots.article_no:
    tv = self.temporal.check_provision(slots.law_short, slots.article_no, as_of)
    trace["temporal"] = tv.to_dict()
    trace["validity_status"] = tv.status
    trace["steps"].append("temporal_check")
    if not tv.is_safe:
        reps = self.temporal.replacement_map(slots.law_short, slots.article_no)
        rep = None
        if reps:
            rep = reps[0]
            trace["replacement"] = reps
        trace["source_url"] = None
        return {"answer": self._compose_warning(tv, rep), "trace": trace,
                "must_show_warning": True, "provision": None}

rows = self.temporal.fetch_current(slots.law_short, slots.article_no)
if not rows:
    trace["steps"].append("miss->C")
    trace["validity_status"] = "查无此条"
    return None
trace["provision_hits"] = len(rows)
trace["source_url"] = rows[0]["source_url"]
trace["steps"].append("provision_render")
# 只保留最新版本的行 (fetch_current 已按 version DESC 排)
latest = rows[0]["version"]
rows = [r for r in rows if r["version"] == latest]
return {"answer": self._render(rows), "trace": trace,
        "must_show_warning": False,
        "provision": [dict(r) for r in rows]}

```

```

# ----- P3 法律效力询问 (D12) -----
if slots.law_short and (slots.law_validity_query or
                        (slots.temporal_query and not slots.topic)):
    tv = self.temporal.check_law(slots.law_short, as_of)
    trace["temporal"] = tv.to_dict()
    trace["validity_status"] = tv.status
    trace["steps"].append("law_validity_check")
    if tv.is_safe:
        ans = (f"《{slots.law_short}》截至 {as_of.isoformat()} 为"
              f"【{tv.status}】，可以继续作为现行依据引用。")
    else:
        ans = tv.warning or ""
        # 该法被废止/修订时，逐条列出最相关的替代条文（含原文与出处）
        reps = self.temporal.replacement_map(slots.law_short)
        if reps:
            trace["replacement"] = reps
            blocks = []
            for rep in reps[:4]:
                rows = self.temporal.fetch_current(
                    rep["law_short"], rep["article_no"])
                if rows:
                    latest = rows[0]["version"]
                    rows = [r for r in rows if r["version"] == latest]
                    blocks.append(f"【现行规定】《{rep['law_short']}》"
                                f"{rows[0]['article_label']}："
                                f"{rows[0]['text']}")
                    if rows[0]["source_url"]:
                        blocks.append(f"（来源：{rows[0]['source_url']}）")
                else:
                    blocks.append(f"【现行规定】《{rep['law_short']}》"
                                f"第{rep['article_no']}条：{rep['note']}")
            ans += "\n\n" + "\n\n".join(blocks)
        if tv.lifecycle_chain and len(tv.lifecycle_chain) > 1:
            ans += f"\n\n【法律沿革】{' → '.join(tv.lifecycle_chain)}"
    return {"answer": ans, "trace": trace, "must_show_warning": not tv.is_safe,
            "provision": None}

# ----- P4 主题条文检索 -----
if slots.topic and slots.provision Seeking:
    rows = self._topic_provisions(slots.topic)
    if rows:
        trace["steps"].append("topic_kw")
        trace["validity_status"] = "现行有效"
        trace["source_url"] = rows[0]["source_url"]
        trace["topic_hits"] = len(rows)
        head = (f"与「{slots.topic}」相关的现行有效条文如下：\n\n")
        return {"answer": head + self._render(rows), "trace": trace,
                "must_show_warning": False,
                "provision": [dict(r) for r in rows]}

```

```

        trace["steps"].append("topic_kw_miss->C")
    return None

def close(self) -> None:
    """关闭**当前线程**的连接（别的线程的连接各自持有，由那条线程自己收尾）。"""
    c = getattr(self._local, "conn", None)
    if c is None:
        return
    try:
        c.close()
    except sqlite3.Error:
        pass
    finally:
        self._local.conn = None

# =====
# 文件: lawgate/channel/b_temporal.py
# =====
# -*- coding: utf-8 -*-
"""通道 B • 时效性状态机（手册 S3.2）。

在手册实现上补三处（docs/deviations.md D14）：
D14.1 ``check_law`` 的 SQL 用 ``from_law LIKE '%法%'`` 会串法：查「公司法」会同时
命中「公司法(2018修正)」与其它含“公司法”的行。改为**精确优先 + 明确后缀变体**
的两级匹配，并对结果按生效日期排序。
D14.2 ``_trace_chain`` 每步只取一行 ``to_law``，链在分支处会断。改为收集全部
后继并按生效日期排序，形成完整沿革链（去重、限深 6）。
D14.3 新增 ``check_law_validity_query``：支持“X法现在还有用吗”这类**无条号**的
时效询问（手册 S3.2 验收用例之一）。
"""

from __future__ import annotations

import re
import sqlite3
import threading
from dataclasses import dataclass, field
from datetime import date

from lawgate.config import get_settings
from lawgate.knowledge.seed_corpus import SUPERSEDE_MAP

@dataclass
class TemporalVerdict:
    status: str
    is_safe: bool
    warning: str | None = None
    replacement: dict | None = None
    lifecycle_chain: list | None = None

```

```
as_of: str = ""
matched_law: str | None = None

def to_dict(self) -> dict:
    return {"status": self.status, "is_safe": self.is_safe,
            "warning": self.warning, "replacement": self.replacement,
            "chain": self.lifecycle_chain, "as_of": self.as_of,
            "matched_law": self.matched_law}
```

```
class TemporalChecker:
    def __init__(self, db_path: str | None = None):
        self.db_path = db_path or get_settings().db_path
        # 连接按线程持有 (D28): sqlite3 连接不能跨线程使用, 而本对象会被
        # FastAPI 线程池 / 评测线程池里的多个线程共享。
        self._local = threading.local()

    @property
    def conn(self) -> sqlite3.Connection:
        """当前线程的连接 (首次访问时建立)。"""
        c = getattr(self._local, "conn", None)
        if c is None:
            c = sqlite3.connect(self.db_path)
            c.row_factory = sqlite3.Row
            self._local.conn = c
        return c

    # ----- 内部
    def _lifecycle_rows(self, law_short: str) -> list[sqlite3.Row]:
        """D14.1: **仅精确匹配** from_law。

        曾经加过“LIKE 变体”兜底, 但那是错的: 查 “公司法” 时会命中
        “公司法(2018修正)” 这条**旧版本**沿革记录, 于是把 2024-07-01 起已生效的
        现行《公司法》误判为“已修订”。查询 “公司法(2018修正)” 时精确匹配即可命中,
        不存在需要模糊匹配的场景。
        """
        return sorted(
            self.conn.execute(
                "SELECT * FROM law_lifecycle WHERE from_law = ?", (law_short,)
            ).fetchall(),
            key=lambda r: r["effective_date"])

    def _trace_chain(self, src: str, max_depth: int = 6) -> list[str]:
        """D14.2: 收集全部后继, 形成完整沿革链。"""
        chain, seen, frontier = [src], {src}, [src]
        while frontier and len(chain) < max_depth:
            nxt: list[str] = []
            for cur in frontier:
                for r in self.conn.execute(
                    "SELECT to_law FROM law_lifecycle WHERE from_law = ? "
```

```

        "AND to_law IS NOT NULL ORDER BY effective_date", (cur,)):
    t = r["to_law"]
    if t and t not in seen:
        seen.add(t)
        chain.append(t)
        nxt.append(t)
    frontier = nxt
    return chain

# ----- 对外
def check_law(self, law_short: str, as_of: date | None = None) -> TemporalVerdict:
    as_of = as_of or date.today()
    matched = None
    for r in self._lifecycle_rows(law_short):
        try:
            eff = date.fromisoformat(r["effective_date"])
        except (TypeError, ValueError):
            continue
        if as_of >= eff:
            abolished = r["relation"] in ("废止", "替代")
            status = "已废止" if abolished else "已修订"
            note = r["note"] or ""
            warning = (f"《{law_short}》已于 {eff.isoformat()} "
                       f"'被废止' if abolished else '被修订', "
                       f"现行规定见《{r['to_law']}》。{note}")
            return TemporalVerdict(
                status=status, is_safe=False, warning=warning,
                replacement={"law_short": r["to_law"], "note": note,
                             "relation": r["relation"]},
                lifecycle_chain=self._trace_chain(law_short),
                as_of=as_of.isoformat(), matched_law=r["from_law"])
        matched = r
    # 未到生效日的修订
    if matched is not None:
        return TemporalVerdict(
            status="尚未生效", is_safe=True,
            warning=(f"提示: 《{law_short}》的"
                    f"{matched['relation']}将于 {matched['effective_date']} 生效, "
                    f"当前仍适用修订前规定。"),
            lifecycle_chain=self._trace_chain(law_short),
            as_of=as_of.isoformat(), matched_law=matched["from_law"])
    return TemporalVerdict(status="现行有效", is_safe=True,
                            as_of=as_of.isoformat(), matched_law=law_short)

def check_provision(self, law_short: str, article_no: int,
                    as_of: date | None = None) -> TemporalVerdict:
    as_of = as_of or date.today()
    # 必须限定 item_no='', 否则"一条多项"时 paragraph_no=1 会命中**项**行,
    # 拿到的 preview 就成了最后一个项而不是本条正文 (修复 D14.4)。
    row = self.conn.execute(

```

```

root = Path(judgments_dir)
conn = sqlite3.connect(db_path)
n_ok = n_skip = 0
causes: dict[str, int] = {}
for f in sorted(root.rglob("*.json")):
    try:
        d = json.loads(f.read_text(encoding="utf-8"))
    except Exception: # noqa: BLE001
        n_skip += 1
        continue
    if isinstance(d, list):
        n_skip += 1
        continue
    full = d.get("full_text") or ""
    case_no = d.get("case_no") or extract_case_no(full)
    if not case_no:
        n_skip += 1
        continue
    fld = case_no_fields(case_no)
    cause = d.get("cause_action") or ""
    causes[cause] = causes.get(cause, 0) + 1
    conn.execute(
        """INSERT OR REPLACE INTO case_registry
        (case_no, year, court_code, court_name, case_type, seq_no, cause_action,
        judgment_date, exists_in_db, source_url, doc_hash, data_source)
        VALUES (?, ?, ?, ?, ?, ?, ?, 1, ?, ?, ?)""",
        (fld["normalized"], fld["year"], fld["court_code"],
        d.get("court_name"), fld["case_type"], fld["seq_no"], cause,
        d.get("judgment_date"), d.get("source_url"),
        hashlib.md5(full.encode("utf-8")).hexdigest(), data_source))
    n_ok += 1
conn.commit()
conn.close()
return {"n_imported": n_ok, "n_skipped": n_skip, "causes": causes}

```

```

def manual_import(db_path: str, rows: list[dict],
                  data_source: str = "MANUAL") -> int:
    """降级路径 (手册 S2.5): 人工逐条录入 [case_no, court_name, cause_action]。"""
    conn = sqlite3.connect(db_path)
    n = 0
    for r in rows:
        fld = case_no_fields(r.get("case_no", ""))
        if not fld["year"]:
            continue
        conn.execute(
            """INSERT OR IGNORE INTO case_registry
            (case_no, year, court_code, court_name, case_type, seq_no, cause_action,
            judgment_date, exists_in_db, source_url, doc_hash, data_source)
            VALUES (?, ?, ?, ?, ?, ?, ?, 1, ?, ?, ?)""",

```

```

        (fld["normalized"], fld["year"], fld["court_code"],
         r.get("court_name"), fld["case_type"], fld["seq_no"],
         r.get("cause_action"), r.get("judgment_date"), None, None,
         data_source))
    n += conn.total_changes and 1 or 0
    conn.commit()
    conn.close()
    return n

# =====
# 文件: lawgate/knowledge/rerank.py
# =====
# -*- coding: utf-8 -*-
"""重排器 (手册 S3.8 的 `rerank=True` 路径)。


手册给的“文档内含 query 关键词加分”过于粗糙：它把 query 当字符集合，对长文档天然有利。这里改用两段式：



1. BM25 (rank_bm25) 在候选集内打分——对“同一法条/同一案由的相近表述”有效；
2. 条款号/案号精确匹配加权——法律检索里“条号命中”必须压过一般词面相似。



最终分数 =  $0.6 \cdot \text{BM25归一}$  +  $0.25 \cdot \text{字符覆盖率}$  +  $0.15 \cdot \text{结构命中 (条号/案号)}$ 。  

中文分词优先 jieba, 缺失则退化为字符二元组。`reranker` 名称写入溯源。


"""

from __future__ import annotations

import re
import warnings

from lawgate.knowledge.judgment_parser import RE_CASE_NO # D31 唯一出处

_ARTICLE_RE = re.compile(r"第([零〇一二三四五六七八九十百千0-9]+)条")

def _case_nos(text: str) -> set[str]:
    """把文本里出现的**每一个**合法案号抽成规范化串 (D31 唯一出处)。


旧实现用本地无捕获组的案号正则 (与判断库口径分叉)；现统一走 judgment_parser。  

RE_CASE_NO 的 5 个捕获组 (年份/法院/类型代字/审级/序号) 逐条重组完整串。


"""
    out: set[str] = set()
    for m in RE_CASE_NO.finditer(text or ""):
        year, court, cat, subcat, seq = m.groups()
        out.add(f"({year}) {court} {cat} {subcat or ''} {int(seq)}号")
    return out

try: # jieba 会触发 pkg_resources 弃用告警，屏蔽之
    warnings.filterwarnings("ignore", category=UserWarning, module="jieba")
    import jieba # type: ignore

```

```
jieba.setLogLevel(60)
_HAS_JIEBA = True
except Exception: # noqa: BLE001
    _HAS_JIEBA = False

def tokenize(text: str) -> list[str]:
    if _HAS_JIEBA:
        return [t for t in jieba.lcut(text) if t.strip()]
    t = "".join(text.split())
    return [t[i:i + 2] for i in range(max(len(t) - 1, 1))] or [t]

class Reranker:
    def __init__(self, use_bm25: bool = True):
        self.use_bm25 = use_bm25
        self.name = ("bm25+char+struct" if use_bm25 else "char+struct")

    def rerank(self, query: str, hits: list, top_n: int = 5) -> list:
        """hits: list[Hit] (含 .text/.meta/.score)。返回重排后的前 top_n 条。"""
        if not hits:
            return []
        docs = [h.text for h in hits]
        bm = self._bm25(query, docs) if self.use_bm25 else [0.0] * len(docs)

        q_chars = set("".join(query.split()))
        q_arts = set(_ARTICLE_RE.findall(query))
        q_cases = set(_case_nos(query)) # D31: 案号抽取走唯一出处 (库口径)

        scored = []
        for i, h in enumerate(hits):
            doc = h.text
            cover = len(q_chars & set("".join(doc.split())) / max(len(q_chars), 1)
            meta = h.meta or {}
            struct = 0.0
            if q_arts:
                doc_arts = set(_ARTICLE_RE.findall(doc))
                lbl = str(meta.get("article", ""))
                doc_arts |= {lbl} if lbl else set()
                if q_arts & doc_arts:
                    struct = 1.0
            if q_cases and any(c in doc for c in q_cases):
                struct = 1.0
            final = 0.60 * bm[i] + 0.25 * cover + 0.15 * struct
            scored.append((final, h))

        scored.sort(key=lambda t: -t[0])
        out = []
        for final, h in scored[:top_n]:
```

```
        h2 = type(h)(h.doc_id, h.text, dict(h.meta or {}), float(final))
        setattr(h2, "vector_score", h.score)
        out.append(h2)
    return out

@staticmethod
def _bm25(query: str, docs: list[str]) -> list[float]:
    try:
        from rank_bm25 import BM25Okapi

        corpus = [tokenize(d) for d in docs]
        if not any(corpus):
            return [0.0] * len(docs)
        bm = BM25Okapi(corpus)
        raw = bm.get_scores(tokenize(query))
        hi = float(max(raw)) if len(raw) else 0.0
        return [float(x) / hi if hi > 0 else 0.0 for x in raw]
    except Exception: # noqa: BLE001
        return [0.0] * len(docs)

# =====
# 文件: lawgate/knowledge/risk_terms.py
# =====
# -*- coding: utf-8 -*-
"""判分用的词表（从 metrics 中拆出，便于法学生独立维护与复核）。

这些词表直接决定 E5 的 TVC 与“非法引用”判定，属于**影响结论的定义**，
因此单独成文件、逐条可审。
"""
from __future__ import annotations

# 失效提示语：句中若出现其一，则“提到失效法名”不算非法引用
ABOLISH_MARKERS = [
    "已废止", "已失效", "已修订", "已被废止", "已被修订", "废止", "失效",
    "不再适用", "已随该法", "已被取代", "已停止适用", "修订前", "旧法",
    "已由", "替代", "已被替代", "不再有效", "已不适用",
]

# 拒答/未作答标记
REFUSAL_MARKERS = [
    "无法确定", "无法回答", "不能确定", "无法给出", "没有足够",
    "建议咨询专业律师", "无法就", "规则兜底后端", "暂无法", "无法判断",
    "我没有", "不了解", "不知道",
]

# 不确定但仍在作答的标记（不算拒答，但纳入 uncertainty 统计）
UNCERTAIN_MARKERS = [
    "可能", "通常", "一般而言", "仅供参考", "不确定", "视情况",
    "建议", "实践中",
]
```

```
]

# 中文停用词 (判分关键词提取用; 只保留影响判分的高频虚词)
STOPWORDS: set[str] = {
    "的", "了", "是", "在", "和", "与", "及", "或", "等", "对", "为", "以", "其",
    "之", "有", "无", "不", "这", "那", "哪", "什么", "哪些", "如何", "怎么",
    "可以", "应当", "应该", "必须", "根据", "依据", "按照", "相关", "有关",
    "规定", "法律", "本条", "该条", "情形", "情况", "问题", "如下", "以下",
    "下列", "如果", "但是", "并且", "而且", "因此", "所以", "由于", "关于",
    "一方", "双方", "对方", "请求", "主张", "人民法院", "法院", "当事人",
    "先生", "女士", "请问", "谢谢", "你好", "需要", "进行", "予以", "加以",
    "上述", "上述的", "本人", "我们", "你们", "他们", "以及", "对于", "而言",
    "是否", "能否", "应该", "可能", "往往", "一般", "通常", "实践", "中",
}

# =====
# 文件: lawgate/knowledge/seed_cases.py
# =====
# -*- coding: utf-8 -*-
"""裁判文书 / 案号真值库的**兜底合成**种子数据 (仅当知识库为空时使用)。

== 状态 ==
- G2 (docs/DATA_GAP.md) 已执行: 生产库 `data/kb/legal_facts.db` 的 case_registry /
  judgments 现由 `scripts/import_judgments.py` 导入的真实裁判文书 (ModelScope
  `qazwsxplkj/cn-judgment-docs`, data_source='CJWS') 填充, 约 1610 条, 全部带真实案号、
  法院、日期、原文与来源链接。
- 本模块的 `generate_cases` / `render_full_text` **仅在知识库为空时才被
  `build_sqlite.build_registry` 调用**, 用于离线联调; `build_registry(keep_real=True)`
  会跳过已存在真实数据的重建。

== 历史披露 (本模块若被使用, 须如实声明) ==
手册 S1.3 要求从中国裁判文书网按 4 类案由抓取 2000 篇真实文书。本环境无法访问文书网时:
* 生成的案号 **全部为合成数据** (data_source='SYNTHETIC'), 遵循真实案号格式规则,
  但不对应真实案件;
* 文书为**程序化生成的案情摘要**, 不含真实裁判说理;
* 此时 E6 结论只能表述为"核验器在格式合法/不存在/案由不符/格式非法四类输入上的
  判别能力", **不能**表述为对真实裁判文书库的覆盖能力。
"""

from __future__ import annotations

import random

# 中级/基层人民法院代字 (真实存在的法院代字, 仅用于构造格式合法的案号)
COURTS: list[tuple[str, str, str]] = [
    ("京01", "北京市第一中级人民法院", "北京"),
    ("京02", "北京市第二中级人民法院", "北京"),
    ("京03", "北京市第三中级人民法院", "北京"),
    ("沪01", "上海市第一中级人民法院", "上海"),
    ("沪02", "上海市第二中级人民法院", "上海"),
```

```
(“粤01”, “广东省广州市中级人民法院”, “广东”),
(“粤03”, “广东省深圳市中级人民法院”, “广东”),
(“浙01”, “浙江省杭州市中级人民法院”, “浙江”),
(“苏01”, “江苏省南京市中级人民法院”, “江苏”),
(“苏05”, “江苏省苏州市中级人民法院”, “江苏”),
(“鲁01”, “山东省济南市中级人民法院”, “山东”),
(“川01”, “四川省成都市中级人民法院”, “四川”),
(“渝01”, “重庆市第一中级人民法院”, “重庆”),
(“津02”, “天津市第二中级人民法院”, “天津”),
(“闽01”, “福建省福州市中级人民法院”, “福建”),
]

CAUSE_ACTIONS = [“民间借贷纠纷”, “劳动争议”, “离婚纠纷”, “房屋租赁合同纠纷”]

CASE_TYPES = [
    ( “民终”, “民事二审”),
    ( “民初”, “民事一审”),
    ( “民再”, “民事再审”),
]

# 每案由的案情摘要模板 (用于生成极简“文书”, 仅供通道 C 语料与 metadata 演示)
FACT_TEMPLATES: dict[str, list[str]] = {
    “民间借贷纠纷”: [
        “原告持有被告出具的借条一份, 载明借款本金{amount}元, 约定月利率{r}%。”
        “被告仅支付部分利息后停止还款, 原告催告未果诉至法院。”
        “双方系朋友关系, 原告通过银行转账向被告支付{amount}元, 未签订书面借款合同, ”
        “亦未约定利息。被告逾期未归还本金, 原告主张自逾期之日起按LPR计算资金占用损失。”
        “借款人向出借人借款{amount}元用于经营周转, 约定借期一年。借期届满后借款人”
        “以经营亏损为由拒不还款, 出借人诉请返还本金及逾期利息。”
    ],
    “劳动争议”: [
        “劳动者于{year}年入职用人单位, 双方未订立书面劳动合同。用人单位以”
        “劳动者严重违反规章制度为由解除劳动关系, 劳动者主张未签合同二倍工资差额”
        “及违法解除赔偿金。”
        “劳动者主张在职期间存在加班事实, 要求用人单位支付加班费。用人单位提供了”
        “考勤记录, 但未就加班审批流程提出反证。”
        “用人单位与劳动者协商一致解除劳动合同, 双方对经济补偿的计算基数与年限存在”
        “争议, 劳动者月工资高于当地上年度职工月平均工资三倍。”
    ],
    “离婚纠纷”: [
        “双方于{year}年登记结婚, 婚后育有一子。因性格不合长期分居, 一方起诉离婚”
        “并要求直接抚养子女, 另一方主张分割婚后共同购置的房产。”
        “一方以感情确已破裂为由起诉离婚, 提出对方存在家庭暴力情形, 请求损害赔偿”
        “并在财产分割中予以照顾。”
        “双方在婚姻登记机关申请离婚登记后, 一方在三十日内撤回申请。另一方随后”
        “直接提起离婚诉讼, 并主张婚前个人财产不予分割。”
    ],
    “房屋租赁合同纠纷”: [
        “承租人与出租人签订房屋租赁合同, 约定租期两年。承租人因工作调动提前退租, ”
        “要求退还押金, 出租人以承租人违约为由拒绝返还。”
    ]
}
```

```
        "租赁期限届满后双方未续签书面合同，承租人继续使用房屋并按月支付租金。"
        "出租人拟收回房屋，承租人主张享有优先承租权。",
        "承租人未经出租人同意将房屋转租给第三人，出租人主张解除租赁合同并要求"
        "承租人赔偿损失。",
    ],
}

JUDGMENT_REASONING: dict[str, str] = {
    "民间借贷纠纷": (
        "本院认为，合法的借贷关系受法律保护。结合借条、转账凭证及双方陈述，"
        "可以认定双方之间存在借贷合意及款项实际交付事实。借款人应当按照约定期限"
        "返还借款。关于利息，出借人主张的利率不得违反国家有关规定，超出部分不予支持。",
    ),
    "劳动争议": (
        "本院认为，用人单位自用工之日起超过一个月不满一年未与劳动者订立书面"
        "劳动合同的，应当向劳动者每月支付二倍的工资。用人单位主张系合法解除，"
        "但未能提交充分证据证明劳动者存在严重违反规章制度的事实，应当承担"
        "举证不能的不利后果。",
    ),
    "离婚纠纷": (
        "本院认为，人民法院审理离婚案件应当进行调解；感情确已破裂、调解无效的，"
        "应当准予离婚。关于子女抚养，应当按照最有利于未成年子女的原则并结合"
        "子女真实意愿确定。夫妻共同财产应当依法分割，并照顾子女、女方和无过错方权益。",
    ),
    "房屋租赁合同纠纷": (
        "本院认为，依法成立的租赁合同对当事人具有法律约束力。承租人无正当理由"
        "未支付或者迟延履行租金的，出租人可以请求其在合理期限内支付。承租人对"
        "租赁物的保管义务及返还义务应当依照合同约定和法律规定履行。",
    ),
}

def generate_cases(n_per_cause: int = 150, seed: int = 42) -> list[dict]:
    """生成确定性的合成案号库。返回 case_registry 行。"""
    rng = random.Random(seed)
    rows: list[dict] = []
    used: set[str] = set()
    seq_counter: dict[tuple, int] = {}

    for cause in CAUSE_ACTIONS:
        made = 0
        guard = 0
        while made < n_per_cause and guard < n_per_cause * 50:
            guard += 1
            code, cname, region = rng.choice(COURTS)
            ctype, ctype_name = rng.choice(CASE_TYPES)
            year = rng.choice([2019, 2020, 2021, 2022, 2023, 2024, 2025])
            key = (year, code, ctype)
            seq_counter[key] = seq_counter.get(key, rng.randint(100, 3000)) + rng.randint(1, 37)
            seq = seq_counter[key]
```

```

        case_no = f"({year}) {code} {ctype} {seq} 号"
        if case_no in used:
            continue
        used.add(case_no)
        month = rng.randint(1, 12)
        day = rng.randint(1, 28)
        jy = year + (1 if ctype in ("民终", "民再") and month > 6 else 0)
        rows.append({
            "case_no": case_no,
            "year": year,
            "court_code": code,
            "court_name": cname,
            "case_type": ctype_name,
            "seq_no": seq,
            "cause_action": cause,
            "judgment_date": f"{jy}-{month:02d}-{day:02d}",
            "exists_in_db": 1,
            "region": region,
            "facts": rng.choice(FACT_TEMPLATES[cause]),
            "reasoning": JUDGMENT_REASONING[cause],
        })
        made += 1
    return rows

def render_full_text(row: dict, rng: random.Random) -> str:
    """渲染一份极简合成判决书(含案号、案由、法院、说理), 供通道 C 语料。"""
    amount = rng.choice(["30000", "50000", "100000", "200000", "450000"])
    facts = row["facts"].format(amount=amount, r=row.get("_rate", 1),
                                year=row["year"] - 2)

    return (
        f"{row['court_name']}\n民事判决书\n{row['case_no']}\n\n"
        f"案由: {row['cause_action']}\n\n"
        f"【原告诉称】{facts}\n\n"
        f"【本院查明】经审理查明, 当事人的陈述与提交的证据能够相互印证, "
        f"本院对案件基本事实予以确认.\n\n"
        f"【本院认为】{row['reasoning']}\n\n"
        f"综上, 依照相关法律规定, 判决如下:\n一、被告于本判决生效之日起十日内"
        f"履行相应义务;\n二、驳回原告的其他诉讼请求.\n\n"
        f"审判长 合成审判员\n审判员 合成审判员\n"
        f"审判员 合成审判员\n{row['judgment_date']}\n"
        f"书记员 合成书记员\n\n"
        f"【合成数据提示】本文书由 CA-LegalGate 项目程序化生成, "
        f"用于离线环境下的系统联调, _不对应任何真实案件_。"
    )

# =====
# 文件: lawgate/knowledge/seed_corpus.py
# =====

```

```
# -*- coding: utf-8 -*-
```

```
"""种子法条语料 (手册 S1.1 的离线降级路径)。
```

```
== 重要说明 (必须随论文/申报书一并披露) ==
```

本模块内置的条文文本由项目组**人工录入**，覆盖“现行有效 / 已修订 / 已废止 / 时效陷阱”四类所需的关键条文，用于在**无干净网络环境**下打通全链路产出实验。每一条的真值状态都是“PENDING_FLK_VERIFICATION”——即：条文内容经人工录入，**尚未**逐字与国家法律法规数据库 (FLK) 官方文本比对。

正式申报前必须执行：

```
python scripts/import_law_text.py --raw-dir data/raw --verify
```

把从 FLK 下载的官方 .docx/.txt 覆盖入库，并把 ingest_provenance_verification 提升为 VERIFIED。届时 S2.4 的“抽验 ≥95%”才有意义。

数据来源标注：

source_url 指向该法在 FLK 的检索页 (可追溯到官方)，而非本次抓取的具体文件。

```
"""
```

```
from __future__ import annotations
```

```
SEED_VERSION = "seed-2026.09"
```

```
# ----- 法源元信息
```

```
LAW_SPECS: dict[str, dict] = {
```

```
    "民法典": {
```

```
        "law_name": "中华人民共和国民法典",
```

```
        "law_level": "法律",
```

```
        "version": "2020",
```

```
        "effective_date": "2021-01-01",
```

```
        "publish_date": "2020-05-28",
```

```
        "validity_status": "现行有效",
```

```
        "source_url": "https://flk.npc.gov.cn/detail2.html?ZmY4MDgxODE3Yjk3MDgxMDEzZGExNzVjYTlkYjQ2Mw==",
```

```
        "text": ""
```

第一编 总则

第一章 基本规定

第一条 为了保护民事主体的合法权益，调整民事关系，维护社会和经济秩序，适应中国特色社会主义发展要求，弘扬社会主义核心价值观，根据宪法，制定本法。

第三条 民事主体的人身权利、财产权利以及其他合法权益受法律保护，任何组织或者个人不得侵犯。

第七条 民事主体从事民事活动，应当遵循诚信原则，秉持诚实，恪守承诺。

第十条 处理民事纠纷，应当依照法律；法律没有规定的，可以适用习惯，但是不得违背公序良俗。

第二章 自然人

第十九条 八周岁以上的未成年人为限制民事行为能力人，实施民事法律行为由其法定代理人代理或者经其法定代理人同意、追认；但是，可以独立实施纯获利益的民事法律行为或者与其年龄、智力相适应的民事法律行为。

第二十条 不满八周岁的未成年人为无民事行为能力人，由其法定代理人代理实施民事法律行为。

第四十条 自然人下落不明满二年的，利害关系人可以向人民法院申请宣告该自然人为失踪人。

第六章 民事法律行为

第一百四十三条 具备下列条件的民事法律行为有效：

(一) 行为人具有相应的民事行为能力；

(二) 意思表示真实；

(三) 不违反法律、行政法规的强制性规定，不违背公序良俗。

第一百四十四条 无民事行为能力人实施的民事法律行为无效。

第一百四十六条 行为人与相对人以虚假的意思表示实施的民事法律行为无效。以虚假的意思表示隐藏的民事法律行为的效力，依照有关法律规定处理。

第一百五十三条 违反法律、行政法规的强制性规定的民事法律行为无效，但是该强制性规定不导致该民事法律行为无效的除外。违背公序良俗的民事法律行为无效。

第一百五十四条 行为人与相对人恶意串通，损害他人合法权益的民事法律行为无效。

第一百五十七条 民事法律行为无效、被撤销或者确定不发生效力后，行为人因该行为取得的财产，应当予以返还；不能返还或者没有必要返还的，应当折价补偿。有过错的一方应当赔偿对方由此所受到的损失；各方都有过错的，应当各自承担相应的责任。法律另有规定的，依照其规定。

第九章 诉讼时效

第一百八十八条 向人民法院请求保护民事权利的诉讼时效期间为三年。法律另有规定的，依照其规定。诉讼时效期间自权利人知道或者应当知道权利受到损害以及义务人之日起计算。法律另有规定的，依照其规定。但是，自权利受到损害之日起超过二十年的，人民法院不予保护，有特殊情况，人民法院可以根据权利人的申请决定延长。

第三编 合同

第一章 一般规定

第四百六十四条 合同是民事主体之间设立、变更、终止民事法律关系的协议。婚姻、收养、监护等有关身份关系的协议，适用有关该身份关系的法律规定；没有规定的，可以根据其性质参照适用本编规定。

第四百六十五条 依法成立的合同，受法律保护。依法成立的合同，仅对当事人具有法律约束力，但是法律另有规定的除外。

第二章 合同的订立

第四百六十九条 当事人订立合同，可以采用书面形式、口头形式或者其他形式。书面形式是合同书、信件、电报、电传、传真等可以有形地表现所载内容的形式。以电子数据交换、电子邮件等方式能够有形地表现所载内容，并可以随时调取查用的数据电文，视为书面形式。

第四百九十条 当事人采用合同书形式订立合同的，自当事人均签名、盖章或者按指印时合同成立。在签名、盖章或者按指印之前，当事人一方已经履行主要义务，对方接受时，该合同成立。

第四百九十六条 格式条款是当事人为了重复使用而预先拟定，并在订立合同时未与对方协商的条款。采用格式条款订立合同的，提供格式条款的一方应当遵循公平原则确定当事人之间的权利和义务，并采取合理的方式提示对方注意免除或者减轻其责任等与对方有重大利害关系的条款，按照对方的要求，对该条款予以说明。

第四百九十七条 有下列情形之一的，该格式条款无效：

- （一）具有本法第一编第六章第三节和本法第五百零六条规定的无效情形；
- （二）提供格式条款一方不合理地免除或者减轻其责任、加重对方责任、限制对方主要权利；
- （三）提供格式条款一方排除对方主要权利。

第四章 合同的履行

第五百零九条 当事人应当按照约定全面履行自己的义务。当事人应当遵循诚信原则，根据合同的性质、目的和交易习惯履行通知、协助、保密等义务。

第五百一十条 合同生效后，当事人就质量、价款或者报酬、履行地点等内容没有约定或者约定不明确的，可以协议补充；不能达成补充协议的，按照合同相关条款或者交易习惯确定。

第七章 合同的权利义务终止

第五百六十三条 有下列情形之一的，当事人可以解除合同：

- （一）因不可抗力致使不能实现合同目的；
- （二）在履行期限届满前，当事人一方明确表示或者以自己的行为表明不履行主要债务；
- （三）当事人一方迟延履行主要债务，经催告后在合理期限内仍未履行；
- （四）当事人一方迟延履行债务或者有其他违约行为致使不能实现合同目的；
- （五）法律规定的其他情形。

以持续履行的债务为内容的不定期合同，当事人可以随时解除合同，但是应当在合理期限之前通知对方。

第八章 违约责任

第五百七十七条 当事人一方不履行合同义务或者履行合同义务不符合约定的，应当承担继续履行、采取补救措施或者赔偿损失等违约责任。

第五百八十四条 当事人一方不履行合同义务或者履行合同义务不符合约定，造成对方损失的，损失赔偿额应当相当于因违约所造成的损失，包括合同履行后可以获得的利益；但是，不得超过违约一方订立合同时预见到或者应当预见到的因违约可能造成的损失。

第五百八十五条 当事人可以约定一方违约时应当根据违约情况向对方支付一定数额的违约金，也可以约定因违约产生的损失赔偿额的计算方法。约定的违约金低于造成的损失的，人民法院或者仲裁机构可以根据当事人的请求予以增加；约定的违约金过分高于造成的损失的

，人民法院或者仲裁机构可以根据当事人的请求予以适当减少。

第五百八十六条 当事人可以约定一方方向对方给付定金作为债权的担保。定金合同自实际交付定金时成立。定金的数额由当事人约定；但是，不得超过主合同标的额的百分之二十，超过部分不产生定金的效力。

第五百八十七条 债务人履行债务的，定金应当抵作价款或者收回。给付定金的一方不履行债务或者履行债务不符合约定，致使不能实现合同目的，无权请求返还定金；收受定金的一方不履行债务或者履行债务不符合约定，致使不能实现合同目的的，应当双倍返还定金。

第十二章 借款合同

第六百六十七条 借款合同是借款人向贷款人借款，到期返还借款并支付利息的合同。

第六百六十八条 借款合同应当采用书面形式，但是自然人之间借款另有约定的除外。借款合同的内容一般包括借款种类、币种、用途、数额、利率、期限和还款方式等条款。

第六百七十四条 借款人应当按照约定的期限支付利息。对支付利息的期限没有约定或者约定不明确，依据本法第五百一十条的规定仍不能确定，借款期间不满一年的，应当在返还借款时一并支付；借款期间一年以上的，应当在每届满一年时支付，剩余期间不满一年的，应当在返还借款时一并支付。

第六百七十五条 借款人应当按照约定的期限返还借款。对借款期限没有约定或者约定不明确，依据本法第五百一十条的规定仍不能确定的，借款人可以随时返还；贷款人可以催告借款人在合理期限内返还。

第六百七十六条 借款人未按照约定的期限返还借款的，应当按照约定或者国家有关规定支付逾期利息。

第六百七十九条 自然人之间的借款合同，自贷款人提供借款时成立。

第六百八十条 禁止高利放贷，借款的利率不得违反国家有关规定。借款合同对支付利息没有约定的，视为没有利息。借款合同对支付利息约定不明确，当事人不能达成补充协议的，按照当地或者当事人的交易方式、交易习惯、市场利率等因素确定利息；自然人之间借款的，视为没有利息。

第十三章 租赁合同

第七百零三条 租赁合同是出租人将租赁物交付承租人使用、收益，承租人支付租金的合同。

第七百零四条 租赁合同的内容一般包括租赁物的名称、数量、用途、租赁期限、租金及其支付期限和方式、租赁物维修等条款。

第七百零五条 租赁期限不得超过二十年。超过二十年的，超过部分无效。租赁期间届满，当事人可以续订租赁合同，但是约定的租赁期限自续订之日起不得超过二十年。

第七百零七条 租赁期限六个月以上的，应当采用书面形式。当事人未采用书面形式，无法确定租赁期限的，视为不定期租赁。

第七百一十四条 承租人应当妥善保管租赁物，因保管不善造成租赁物毁损、灭失的，应当承担赔偿责任。

第七百一十六条 承租人经出租人同意，可以将租赁物转租给第三人。承租人转租的，承租人与出租人之间的租赁合同继续有效；第三人造成租赁物损失的，承租人应当赔偿损失。承租人未经出租人同意转租的，出租人可以解除合同。

第七百二十一条 承租人应当按照约定的期限支付租金。对支付租金的期限没有约定或者约定不明确，依据本法第五百一十条的规定仍不能确定，租赁期限不满一年的，应当在租赁期限届满时支付；租赁期限一年以上的，应当在每届满一年时支付，剩余期限不满一年的，应当在租赁期限届满时支付。

第七百二十二条 承租人无正当理由未支付或者迟延支付租金的，出租人可以请求承租人在合理期限内支付；承租人逾期不支付的，出租人可以解除合同。

第七百二十八条 出租人未通知承租人或者有其他妨害承租人行使优先购买权情形的，承租人可以请求出租人承担赔偿责任。但是，出租人与第三人订立的房屋买卖合同的效力不受影响。

第七百三十三条 租赁期限届满，承租人应当返还租赁物。返还的租赁物应当符合按照约定或者根据租赁物的性质使用后的状态。

第五编 婚姻家庭

第一千零四十二条 禁止包办、买卖婚姻和其他干涉婚姻自由的行为。禁止借婚姻索取财物。禁止重婚。禁止有配偶者与他人同居。禁止家庭暴力。禁止家庭成员间的虐待和遗弃。

第一千零六十二条 夫妻在婚姻关系存续期间所得的下列财产，为夫妻共同财产，归夫妻共同所有：

- (一) 工资、奖金、劳务报酬；
- (二) 生产、经营、投资的收益；
- (三) 知识产权的收益；
- (四) 继承或者受赠的财产，但是本法第一千零六十三条第三项规定的除外；
- (五) 其他应当归共同所有的财产。

夫妻对共同财产，有平等的处理权。

第一千零六十三条 下列财产为夫妻一方的个人财产：

- (一) 一方的婚前财产；
- (二) 一方因受到人身损害获得的赔偿或者补偿；
- (三) 遗嘱或者赠与合同中确定只归一方的财产；

（四）一方专用的生活用品；

（五）其他应当归一方的财产。

第一千零六十四条 夫妻双方共同签名或者夫妻一方事后追认等共同意思表示所负的债务，以及夫妻一方在婚姻关系存续期间以个人名义为家庭日常生活需要所负的债务，属于夫妻共同债务。夫妻一方在婚姻关系存续期间以个人名义超出家庭日常生活需要所负的债务，不属于夫妻共同债务；但是，债权人能够证明该债务用于夫妻共同生活、共同生产经营或者基于夫妻双方共同意思表示的除外。

第一千零七十六条 夫妻双方自愿离婚的，应当签订书面离婚协议，并亲自到婚姻登记机关申请离婚登记。离婚协议应当载明双方自愿离婚的意思表示和对子女抚养、财产以及债务处理等事项协商一致的意见。

第一千零七十七条 自婚姻登记机关收到离婚登记申请之日起三十日内，任何一方不愿意离婚的，可以向婚姻登记机关撤回离婚登记申请。前款规定期限届满后三十日内，双方应当亲自到婚姻登记机关申请发给离婚证；未申请的，视为撤回离婚登记申请。

第一千零七十九条 夫妻一方要求离婚的，可以由有关组织进行调解或者直接向人民法院提起离婚诉讼。人民法院审理离婚案件，应当进行调解；如果感情确已破裂，调解无效的，应当准予离婚。有下列情形之一，调解无效的，应当准予离婚：

（一）重婚或者与他人同居；

（二）实施家庭暴力或者虐待、遗弃家庭成员；

（三）有赌博、吸毒等恶习屡教不改；

（四）因感情不和分居满二年；

（五）其他导致夫妻感情破裂的情形。

一方被宣告失踪，另一方提起离婚诉讼的，应当准予离婚。经人民法院判决不准离婚后，双方又分居满一年，一方再次提起离婚诉讼的，应当准予离婚。

第一千零八十四条 父母与子女间的关系，不因父母离婚而消除。离婚后，子女无论由父或者母直接抚养，仍是父母双方的子女。离婚后，父母对于子女仍有抚养、教育、保护的權利和义务。离婚后，不满两周岁的子女，以由母亲直接抚养为原则。已满两周岁的子女，父母双方对抚养问题协议不成的，由人民法院根据双方的具体情况，按照最有利于未成年子女的原则判决。子女已满八周岁的，应当尊重其真实意愿。

第一千零八十七条 离婚时，夫妻的共同财产由双方协议处理；协议不成的，由人民法院根据财产的具体情况，照顾子女、女方和无过错方权益的原则判决。对夫或者妻在家庭土地承包经营中享有的权益等，应当依法予以保护。

第一千零八十八条 夫妻一方因抚育子女、照料老年人、协助另一方工作等负担较多义务的，离婚时有权向另一方请求补偿，另一方应当给予补偿。具体办法由双方协议；协议不成的，由人民法院判决。

第一千零九十一条 有下列情形之一，导致离婚的，无过错方有权请求损害赔偿：

（一）重婚；

（二）与他人同居；

（三）实施家庭暴力；

（四）虐待、遗弃家庭成员；

（五）有其他重大过错。

第七编 侵权责任

第一千一百六十五条 行为人因过错侵害他人民事权益造成损害的，应当承担侵权责任。依照法律规定推定行为人有过错，其不能证明自己没有过错的，应当承担侵权责任。

第一千一百六十六条 行为人造成他人民事权益损害，不论行为人有无过错，法律规定应当承担侵权责任的，依照其规定。

第一千一百七十九条 侵害他人造成人身损害的，应当赔偿医疗费、护理费、交通费、营养费、住院伙食补助费等为治疗和康复支出的合理费用，以及因误工减少的收入。造成残疾的，还应当赔偿辅助器具费和残疾赔偿金；造成死亡的，还应当赔偿丧葬费和死亡赔偿金。

第一千一百八十三条 侵害自然人人身权益造成严重精神损害的，被侵权人有权请求精神损害赔偿。

第一千一百九十一条 用人单位的工作人员因执行工作任务造成他人损害的，由用人单位承担侵权责任。用人单位承担侵权责任后，可以向有故意或者重大过失的工作人员追偿。劳务派遣期间，被派遣的工作人员因执行工作任务造成他人损害的，由接受劳务派遣的用工单位承担侵权责任；劳务派遣单位有过错的，承担相应的责任。

第一千二百五十八条 在公共道路上、公共场所或者道路上挖掘、修缮安装地下设施等造成他人损害，施工人不能证明已经设置明显标志和采取安全措施的，应当承担侵权责任。窨井等地下设施造成他人损害，管理人不能证明尽到管理职责的，应当承担侵权责任。

"""

},

“合同法”：{

 “law_name”：“中华人民共和国合同法”，

 “law_level”：“法律”，

```
    "version": "1999",
    "effective_date": "1999-10-01",
    "publish_date": "1999-03-15",
    "validity_status": "已废止",
    "source_url": "https://flk.npc.gov.cn/",
    "text": ""
```

总则

第二章 合同的订立

第五十二条 有下列情形之一的，合同无效：

- （一）一方以欺诈、胁迫的手段订立合同，损害国家利益；
- （二）恶意串通，损害国家、集体或者第三人利益；
- （三）以合法形式掩盖非法目的；
- （四）损害社会公共利益；
- （五）违反法律、行政法规的强制性规定。

第三章 合同的效力

第五十八条 合同无效或者被撤销后，因该合同取得的财产，应当予以返还；不能返还或者没有必要返还的，应当折价补偿。有过错的一方应当赔偿对方因此所受到的损失，双方都有过错的，应当各自承担相应的责任。

第七章 违约责任

第一百零七条 当事人一方不履行合同义务或者履行合同义务不符合约定的，应当承担继续履行、采取补救措施或者赔偿损失等违约责任。

第一百一十四条 当事人可以约定一方违约时应当根据违约情况向对方支付一定数额的违约金，也可以约定因违约产生的损失赔偿额的计算方法。约定的违约金低于造成的损失的，当事人可以请求人民法院或者仲裁机构予以增加；约定的违约金过分高于造成的损失的，当事人可以请求人民法院或者仲裁机构予以适当减少。

分则

第十二章 借款合同

第一百九十六条 借款合同是借款人向贷款人借款，到期返还借款并支付利息的合同。

第二百一十一条 自然人之间的借款合同对支付利息没有约定或者约定不明确的，视为不支付利息。自然人之间的借款合同约定支付利息的，借款的利率不得违反国家有关限制借款利率的规定。

```
""",
    },
```

```
"公司法": {
    "law_name": "中华人民共和国公司法",
    "law_level": "法律",
    "version": "2023修订",
    "effective_date": "2024-07-01",
    "publish_date": "2023-12-29",
    "validity_status": "现行有效",
    "source_url": "https://flk.npc.gov.cn/",
    "text": ""
```

第一章 总则

第一条 为了规范公司的组织和行为，保护公司、股东、职工和债权人的合法权益，完善中国特色现代企业制度，弘扬企业家精神，维护社会经济秩序，促进社会主义市场经济的发展，根据宪法，制定本法。

第二章 公司登记

第三章 有限责任公司的设立和组织机构

第四十七条 有限责任公司的注册资本为在公司登记机关登记的全体股东认缴的出资额。全体股东认缴的出资额由股东按照公司章程的规定自公司成立之日起五年内缴足。法律、行政法规以及国务院决定对有限责任公司注册资本实缴、注册资本最低限额、股东出资期限有另行规定的，适用其规定。

第四十九条 股东应当按期足额缴纳公司章程规定的各自所认缴的出资额。股东以货币出资的，应当将货币出资足额存入有限责任公司在银行开设的账户；以非货币财产出资的，应当依法办理其财产权的转移手续。股东未按期足额缴纳出资的，除应当向公司足额缴纳外，还

应当对给公司造成的损失承担赔偿责任。

第五十一条 有限责任公司成立后，董事会应当对股东的出资情况进行核查，发现股东未按期足额缴纳公司章程规定的出资的，应当由公司向该股东发出书面催缴书，催缴出资。未及时履行前款规定的义务，给公司造成损失的，负有责任的董事应当承担赔偿责任。

第五十二条 股东未按照公司章程规定的出资日期缴纳出资，公司依照前条第一款规定发出书面催缴书催缴出资的，可以载明缴纳出资的宽限期；宽限期自公司发出催缴书之日起，不得少于六十日。宽限期届满，股东仍未履行出资义务的，公司经董事会决议可以向该股东发出失权通知，通知应当以书面形式发出。自通知发出之日起，该股东丧失其未缴纳出资的股权。

第五十三条 公司成立后，股东不得抽逃出资。违反前款规定的，股东应当返还抽逃的出资；给公司造成损失的，负有责任的董事、监事、高级管理人员应当与该股东承担连带赔偿责任。

第四章 有限责任公司的股权转让

第八十八条 股东转让已认缴出资但未届出资期限的股权的，由受让人承担缴纳该出资的义务；受让人未按期足额缴纳出资的，转让人对受让人未按期缴纳的出资承担补充责任。未按照公司章程规定的出资日期缴纳出资或者作为出资的非货币财产的实际价额显著低于所认缴的出资额的股东转让股权的，转让人与受让人在出资不足的范围内承担连带责任；受让人不知道且不应当知道存在上述情形的，由转让人承担责任。

第九章 公司债券

第十章 公司财务、会计

第十一章 公司合并、分立、增资、减资

第十二章 公司解散和清算

第十三章 外国公司的分支机构

第十四章 法律责任

第十五章 附则

```
"""
    },

    "公司法(2018修正)": {
        "law_name": "中华人民共和国公司法",
        "law_level": "法律",
        "version": "2018修正",
        "effective_date": "2018-10-26",
        "publish_date": "2018-10-26",
        "validity_status": "已修订",
        "source_url": "https://flk.npc.gov.cn/",
        "text": """
第三章 有限责任公司的设立和组织机构
第二十六条 有限责任公司的注册资本为在公司登记机关登记的全体股东认缴的出资额。法律、行政法规以及国务院决定对有限责任公司注册资本实缴、注册资本最低限额另有规定的，适用其规定。
"""
    },

    "劳动合同法": {
        "law_name": "中华人民共和国劳动合同法",
        "law_level": "法律",
        "version": "2012修正",
        "effective_date": "2013-07-01",
        "publish_date": "2012-12-28",
        "validity_status": "现行有效",
        "source_url": "https://flk.npc.gov.cn/",
        "text": """
第二章 劳动合同的订立
第十条 建立劳动关系，应当订立书面劳动合同。已建立劳动关系，未同时订立书面劳动合同的，应当自用工之日起一个月内订立书面劳动合同。用人单位与劳动者在用工前订立劳动合同的，劳动关系自用工之日起建立。
"""
    }
```

第四章 劳动合同的解除和终止

第三十八条 用人单位有下列情形之一的，劳动者可以解除劳动合同：

- （一）未按照劳动合同约定提供劳动保护或者劳动条件的；
- （二）未及时足额支付劳动报酬的；
- （三）未依法为劳动者缴纳社会保险费的；
- （四）用人单位的规章制度违反法律、法规的规定，损害劳动者权益的；
- （五）因本法第二十六条第一款规定的情形致使劳动合同无效的；
- （六）法律、行政法规规定劳动者可以解除劳动合同的其他情形。

第三十九条 劳动者有下列情形之一的，用人单位可以解除劳动合同：

- （一）在试用期间被证明不符合录用条件的；
- （二）严重违反用人单位的规章制度的；
- （三）严重失职，营私舞弊，给用人单位造成重大损害的。

第四十六条 有下列情形之一的，用人单位应当向劳动者支付经济补偿：

- （一）劳动者依照本法第三十八条规定解除劳动合同的；
- （二）用人单位依照本法第三十六条规定向劳动者提出解除劳动合同并与劳动者协商一致解除劳动合同的；
- （三）用人单位依照本法第四十条规定解除劳动合同的；
- （四）用人单位依照本法第四十一条第一款规定解除劳动合同的。

第四十七条 经济补偿按劳动者在本单位工作的年限，每满一年支付一个月工资的标准向劳动者支付。六个月以上不满一年的，按一年计算；不满六个月的，向劳动者支付半个月工资的经济补偿。劳动者月工资高于用人单位所在直辖市、设区的市级人民政府公布的本地区上年度职工月平均工资三倍的，向其支付经济补偿的标准按职工月平均工资三倍的数额支付，向其支付经济补偿的年限最高不超过十二年。本条所称月工资是指劳动者在劳动合同解除或者终止前十二个月的平均工资。

第七章 法律责任

第八十二条 用人单位自用工之日起超过一个月不满一年未与劳动者订立书面劳动合同的，应当向劳动者每月支付二倍的工资。用人单位违反本法规定不与劳动者订立无固定期限劳动合同的，自应当订立无固定期限劳动合同之日起向劳动者每月支付二倍的工资。

第八十七条 用人单位违反本法规定解除或者终止劳动合同的，应当依照本法第四十七条规定的经济补偿标准的二倍向劳动者支付赔偿金

```
。
"""
    },

    "民事诉讼法": {
        "law_name": "中华人民共和国民事诉讼法",
        "law_level": "法律",
        "version": "2023修正",
        "effective_date": "2024-01-01",
        "publish_date": "2023-09-01",
        "validity_status": "现行有效",
        "source_url": "https://flk.npc.gov.cn/",
        "text": ""
    }
```

第一编 总则

第一章 任务、适用范围和基本原则

第十二章 起诉与受理

第一百二十二条 起诉必须符合下列条件：

- （一）原告是与本案有直接利害关系的公民、法人和其他组织；
- （二）有明确的被告；
- （三）有具体的诉讼请求和事实、理由；
- （四）属于人民法院受理民事诉讼的范围和受诉人民法院管辖。

```
"""
    },

    "民法典时间效力规定": {
```

```

"law_name": "最高人民法院关于适用《中华人民共和国民法典》时间效力的若干规定",
"law_level": "司法解释",
"version": "2020",
"effective_date": "2021-01-01",
"publish_date": "2020-12-29",
"validity_status": "现行有效",
"source_url": "https://www.court.gov.cn/",
"text": ""

```

第一条 民法典施行后的法律事实引起的民事纠纷案件,适用民法典的规定。民法典施行前的法律事实引起的民事纠纷案件,适用当时的法律、司法解释的规定,但是法律、司法解释另有规定的除外。民法典施行前的法律事实持续至民法典施行后,该法律事实引起的民事纠纷案件,适用民法典的规定,但是法律、司法解释另有规定的除外。

第二条 民法典施行前的法律事实引起的民事纠纷案件,当时的法律、司法解释有规定,适用当时的法律、司法解释的规定,但是适用民法典的规定更有利于保护民事主体合法权益,更有利于维护社会和经济秩序,更有利于弘扬社会主义核心价值观的除外。

第三条 民法典施行前的法律事实引起的民事纠纷案件,当时的法律、司法解释没有规定而民法典有规定的,可以适用民法典的规定,但是明显减损当事人合法权益、增加当事人法定义务或者背离当事人合理预期的除外。

```

""
},

```

```

"物权法": {
  "law_name": "中华人民共和国物权法",
  "law_level": "法律", "version": "2007",
  "effective_date": "2007-10-01", "publish_date": "2007-03-16",
  "validity_status": "已废止", "source_url": "https://flk.npc.gov.cn/",
  "text": ""

```

第一编 总则

第一章 基本原则

第九条 不动产物权的设立、变更、转让和消灭,经依法登记,发生法律效力;未经登记,不发生法律效力,但法律另有规定的除外。

第二编 所有权

第四章 一般规定

第三十九条 所有权人对自己的不动产或者动产,依法享有占有、使用、收益和处分的权利。

```

""
},

```

```

"担保法": {
  "law_name": "中华人民共和国担保法",
  "law_level": "法律", "version": "1995",
  "effective_date": "1995-10-01", "publish_date": "1995-06-30",
  "validity_status": "已废止", "source_url": "https://flk.npc.gov.cn/",
  "text": ""

```

第二章 保证

第一节 保证和保证人

第十六条 保证的方式有:

- (一) 一般保证;
- (二) 连带责任保证。

第十九条 当事人对保证方式没有约定或者约定不明确的,按照连带责任保证承担保证责任。

```

""
},

```

```

"婚姻法": {
  "law_name": "中华人民共和国婚姻法",

```

```
    "law_level": "法律", "version": "2001修正",
    "effective_date": "2001-04-28", "publish_date": "2001-04-28",
    "validity_status": "已废止", "source_url": "https://flk.npc.gov.cn/",
    "text": ""
  },
  "侵权责任法": {
    "law_name": "中华人民共和国侵权责任法",
    "law_level": "法律", "version": "2009",
    "effective_date": "2010-07-01", "publish_date": "2009-12-26",
    "validity_status": "已废止", "source_url": "https://flk.npc.gov.cn/",
    "text": ""
  }
},
```

第四章 离婚

第三十二条 男女一方要求离婚的，可由有关部门进行调解或直接向人民法院提出离婚诉讼。人民法院审理离婚案件，应当进行调解；如感情确已破裂，调解无效，应准予离婚。

```
    "text": ""
  },
```

```
  "侵权责任法": {
    "law_name": "中华人民共和国侵权责任法",
    "law_level": "法律", "version": "2009",
    "effective_date": "2010-07-01", "publish_date": "2009-12-26",
    "validity_status": "已废止", "source_url": "https://flk.npc.gov.cn/",
    "text": ""
  }
},
```

第一章 一般规定

第六条 行为人因过错侵害他人民事权益，应当承担侵权责任。根据法律规定推定行为人有过错，行为人不能证明自己没有过错的，应当承担侵权责任。

```
    "text": ""
  },
```

```
  "继承法": {
    "law_name": "中华人民共和国继承法",
    "law_level": "法律", "version": "1985",
    "effective_date": "1985-10-01", "publish_date": "1985-04-10",
    "validity_status": "已废止", "source_url": "https://flk.npc.gov.cn/",
    "text": ""
  }
},
```

第二章 法定继承

第十条 遗产按照下列顺序继承：第一顺序：配偶、子女、父母。第二顺序：兄弟姐妹、祖父母、外祖父母。

```
    "text": ""
  },
```

```
  "收养法": {
    "law_name": "中华人民共和国收养法",
    "law_level": "法律", "version": "1998修正",
    "effective_date": "1999-04-01", "publish_date": "1998-11-04",
    "validity_status": "已废止", "source_url": "https://flk.npc.gov.cn/",
    "text": ""
  }
},
```

第二章 收养关系的成立

第六条 收养人应当同时具备下列条件：

- （一）无子女；
- （二）有抚养教育被收养人的能力；
- （三）未患有在医学上认为不应当收养子女的疾病；
- （四）年满三十周岁。

```
    "text": ""
  },
}
```

```
# 本节是**权威口径**：LAW_SPECS[*]["text"] 中的条文只是离线降级用的旧种子，
# 2026-09-13 完成“真实数据替换”后，`legal_facts.db` 的正文一律来自
# `data/raw/<law_short>.txt`，而该文件由下列脚本从下列真实来源抓取：
#   • scripts/fetch_laws_dataset.py —— 开源结构化数据集 laws-data (MIT)
#   • scripts/fetch_official_repealed.py —— 中国人大网官方法律全文页
# 导入后 ingest_provenance.source_kind / source_url 取这里的值，
# 不再写死为 FLK_OFFICIAL（此前那条记录与实际来源不符，属于必须修正的失真）。
#
# 来源等级：
#   A 中国人大网官方全文页 (www.npc.gov.cn)
#   B 开源结构化数据集 laws-data (原始文本源自官方公开渠道，MIT 许可)
LAWS_DATA_REPO = "https://github.com/13098806890/laws-data/blob/main/"
_NPC_PAGE = "http://www.npc.gov.cn/zgrdw/npc/lfzt/rlyw/2016-07/01/content_{}.htm"

PROVENANCE: dict[str, dict[str, str]] = {
    # ---- 现行有效法：来自 laws-data 结构化数据集 ----
    "民法典": {"source_kind": "LAWS_DATA_DATASET",
                "source_url": LAWS_DATA_REPO + "json/法律/中华人民共和国民法典_20200528.json"},
    "公司法": {"source_kind": "LAWS_DATA_DATASET",
                "source_url": LAWS_DATA_REPO + "json/法律/中华人民共和国公司法_20231229.json"},
    "公司法(2018修正)": {"source_kind": "LAWS_DATA_DATASET",
                           "source_url": LAWS_DATA_REPO + "json/法律/中华人民共和国公司法_20181026.json"},
    "劳动合同法": {"source_kind": "LAWS_DATA_DATASET",
                    "source_url": LAWS_DATA_REPO + "json/法律/中华人民共和国劳动合同法_20121228.json"},
    "民事诉讼法": {"source_kind": "LAWS_DATA_DATASET",
                    "source_url": LAWS_DATA_REPO + "json/法律/中华人民共和国民事诉讼法_20230901.json"},
    "民法典时间效力规定": {
        "source_kind": "LAWS_DATA_DATASET",
        "source_url": LAWS_DATA_REPO +
            "json/司法解释/最高人民法院关于适用《中华人民共和国民法典》时间效力的若干规定_20201229.json"},
    # ---- 已废止法：来自中国人大网官方全文页（原种子仅 1-8 条，现为全文）----
    "合同法": {"source_kind": "NPC_OFFICIAL", "source_url": _NPC_PAGE.format(1992739)},
    "物权法": {"source_kind": "NPC_OFFICIAL", "source_url": _NPC_PAGE.format(1992736)},
    "侵权责任法": {"source_kind": "NPC_OFFICIAL", "source_url": _NPC_PAGE.format(1992753)},
    "婚姻法": {"source_kind": "NPC_OFFICIAL", "source_url": _NPC_PAGE.format(1992746)},
    "继承法": {"source_kind": "NPC_OFFICIAL", "source_url": _NPC_PAGE.format(1992749)},
    "收养法": {"source_kind": "NPC_OFFICIAL", "source_url": _NPC_PAGE.format(1992751)},
    "担保法": {"source_kind": "NPC_OFFICIAL", "source_url": _NPC_PAGE.format(1992740)},
}

# 官方公布条数（用于导入校验；已由官方页面逐条核对）
OFFICIAL_ARTICLE_COUNT: dict[str, int] = {
    "民法典": 1260, "合同法": 428, "物权法": 247, "侵权责任法": 92,
    "婚姻法": 51, "继承法": 37, "收养法": 34, "担保法": 96,
    "公司法": 266, "公司法(2018修正)": 218, "劳动合同法": 98,
    "民事诉讼法": 306, "民法典时间效力规定": 28,
}

for _k, _v in PROVENANCE.items():
    LAW_SPECS[_k].update(_v)
```

```
# ----- 沿革种子
LIFECYCLE_SEED: list[tuple] = [
    ("合同法", "民法典", "废止", "2021-01-01", "主席令第四十五号",
     "民法典施行同时废止合同法、物权法、侵权责任法、婚姻法、继承法、收养法、担保法"),
    ("物权法", "民法典", "废止", "2021-01-01", "主席令第四十五号", None),
    ("侵权责任法", "民法典", "废止", "2021-01-01", "主席令第四十五号", None),
    ("婚姻法", "民法典", "废止", "2021-01-01", "主席令第四十五号", None),
    ("继承法", "民法典", "废止", "2021-01-01", "主席令第四十五号", None),
    ("收养法", "民法典", "废止", "2021-01-01", "主席令第四十五号", None),
    ("担保法", "民法典", "废止", "2021-01-01", "主席令第四十五号", None),
    ("公司法(2018修正)", "公司法", "修订", "2024-07-01",
     "全国人大常委会公告",
     "2023年修订：注册资本认缴出资五年内缴足（第47条）、董事催缴义务（第51条）、"
     "股东失权制度（第52条）、股权转让出资责任（第88条）"),
]
```

已废止法律 → 民法典替代条文映射（用于通道 B 的“现行规定”指引）

```
SUPERSEDE_MAP: dict[str, list[dict]] = {
    "合同法": [
        {"old": "第52条", "new_law": "民法典", "new_article": 143,
         "note": "合同无效事由改由民法典第143、144、146、153、154条规定"},
        {"old": "第52条", "new_law": "民法典", "new_article": 153,
         "note": "违反强制性规定、违背公序良俗的民事法律行为无效"},
        {"old": "第58条", "new_law": "民法典", "new_article": 157,
         "note": "无效后的返还与折价补偿规则"},
        {"old": "第107条", "new_law": "民法典", "new_article": 577,
         "note": "违约责任承担方式"},
        {"old": "第114条", "new_law": "民法典", "new_article": 585,
         "note": "违约金的约定与司法调整"},
        {"old": "第196条", "new_law": "民法典", "new_article": 667,
         "note": "借款合同定义"},
        {"old": "第211条", "new_law": "民法典", "new_article": 680,
         "note": "自然人之间借款利息规则与禁止高利放贷"},
    ],
    "婚姻法": [
        {"old": "第32条", "new_law": "民法典", "new_article": 1079,
         "note": "诉讼离婚与感情破裂的法定情形"},
    ],
    "侵权责任法": [
        {"old": "第6条", "new_law": "民法典", "new_article": 1165,
         "note": "过错责任与过错推定"},
    ],
    "物权法": [
        {"old": "第9条", "new_law": "民法典", "new_article": 209,
         "note": "不动产物权登记生效（民法典物权编）"},
    ],
    "担保法": [
        {"old": "第19条", "new_law": "民法典", "new_article": 686,
```

```
        "note": "约定不明时按一般保证责任承担责任（与担保法相反）"},
    ],
    "继承法": [
        {"old": "第10条", "new_law": "民法典", "new_article": 1127,
         "note": "法定继承人范围与顺序"},
    ],
    "收养法": [
        {"old": "第6条", "new_law": "民法典", "new_article": 1098,
         "note": "收养人条件"},
    ],
    "公司法(2018修正)": [
        {"old": "第26条", "new_law": "公司法", "new_article": 47,
         "note": "注册资本认缴出资须自公司成立之日起五年内缴足（2024-07-01 起）"},
    ],
}
```

----- 别名词典

```
LAW_ALIAS_SEED: dict[str, str] = {
    "民法典": "民法典", "民法": "民法典", "中华人民共和国民法典": "民法典",
    "合同法": "合同法",
    "公司法": "公司法", "新公司法": "公司法",
    "劳动合同法": "劳动合同法", "劳动法": "劳动合同法",
    "消费者权益保护法": "消保法", "消法": "消保法",
    "刑法": "刑法", "治安管理处罚法": "治安管理处罚法",
    "物权法": "物权法", "担保法": "担保法", "婚姻法": "婚姻法",
    "侵权责任法": "侵权责任法", "继承法": "继承法", "收养法": "收养法",
    "民事诉讼法": "民事诉讼法", "民诉法": "民事诉讼法",
    "民法典时间效力": "民法典时间效力规定",
    "时间效力规定": "民法典时间效力规定",
    "民法典时间效力规定": "民法典时间效力规定",
}
```

----- 主题词典

```
TOPIC_KEYWORDS_SEED: dict[str, list[str]] = {
    "民间借贷": ["借钱", "借款", "欠钱", "还钱", "借条", "欠条", "利息", "民间借贷",
                 "高利贷", "逾期利息", "本金", "不还", "不还款", "欠款", "赖账",
                 "借我", "还没还", "催款"],
    "劳动争议": ["加班费", "辞退", "裁员", "工伤", "社保", "劳动合同", "竞业限制",
                 "经济补偿", "赔偿金", "试用期", "未签劳动合同", "被辞退",
                 "无故开除", "离职补偿", "劳动纠纷", "劳动仲裁", "劳动争议"],
    "离婚纠纷": ["离婚", "抚养权", "抚养费", "夫妻共同财产", "离婚冷静期", "彩礼",
                 "感情破裂", "共同债务", "婚前财产", "分居", "起诉离婚"],
    "房屋租赁": ["押金", "退租", "房东", "租客", "租赁合同", "涨房租", "转租",
                 "不定期租赁", "租金", "提前退租", "不退押金"],
    "违约金": ["违约金", "定金", "订金", "赔偿金", "过高", "调整", "双倍返还",
               "违约赔偿"],
    "合同效力": ["合同无效", "无效合同", "格式条款", "欺诈", "胁迫", "公序良俗",
                 "强制性规定", "恶意串通", "合同有效", "合同成立"],
    "侵权责任": ["侵权", "过错责任", "人身损害", "精神损害", "赔偿责任",
                 "用人单位责任", "损害赔偿", "受伤"],
}
```

```
    "公司治理": ["注册资本", "认缴", "实缴", "抽逃出资", "股东出资", "董事会",
                  "股权转让", "失权", "出资期限"],
}

# =====
# 文件: lawgate/knowledge/store.py
# =====
# -*- coding: utf-8 -*-
"""向量库封装（手册 S2.6 / 通道 C 的检索后端）。

两条后端：
    * ``ChromaStore``——chromadb 持久化集合（手册指定）；
    * ``NumpyStore``——npz + jsonl 的精确余弦检索（零依赖兜底）。

环境实测中 chromadb 可导入，但离线/受限环境下其持久化与遥测可能失败，
故 ``get_store()`` 做可用性探测并自动降级，且把 ``backend`` 写进溯源。
"""

from __future__ import annotations

import json
from dataclasses import dataclass
from pathlib import Path

import numpy as np

@dataclass
class Hit:
    doc_id: str
    text: str
    meta: dict
    score: float # 余弦相似度（越大越相关）

class BaseStore:
    backend: str = "base"
    dim: int = 0

    def add(self, ids, docs, metas, embeddings) -> None: # pragma: no cover
        raise NotImplementedError

    def query(self, embedding: np.ndarray, top_k: int = 8) -> list[Hit]: # pragma: no cover
        raise NotImplementedError

    def count(self) -> int: # pragma: no cover
        raise NotImplementedError

class NumpyStore(BaseStore):
```

```
"""精确余弦检索。数据量大时慢，但本文档级规模（数千条）完全够用且完全可控。"""
```

```
backend = "numpy"
```

```
def __init__(self, path: str | Path):
    self.path = Path(path)
    self.path.mkdir(parents=True, exist_ok=True)
    self._emb = np.zeros((0, 0), dtype=np.float32)
    self._ids: list[str] = []
    self._docs: list[str] = []
    self._metas: list[dict] = []
    self.load()
```

```
# ----- 持久化
```

```
@property
```

```
def _emb_file(self) -> Path:
    return self.path / "embeddings.npy"
```

```
@property
```

```
def _meta_file(self) -> Path:
    return self.path / "records.jsonl"
```

```
def save(self) -> None:
    np.save(self._emb_file, self._emb)
    with open(self._meta_file, "w", encoding="utf-8") as f:
        for i, d, m in zip(self._ids, self._docs, self._metas):
            f.write(json.dumps({"id": i, "doc": d, "meta": m},
                               ensure_ascii=False) + "\n")
```

```
def load(self) -> bool:
    if not (self._emb_file.exists() and self._meta_file.exists()):
        return False
    self._emb = np.load(self._emb_file).astype(np.float32)
    self._ids, self._docs, self._metas = [], [], []
    with open(self._meta_file, encoding="utf-8") as f:
        for line in f:
            if not line.strip():
                continue
            r = json.loads(line)
            self._ids.append(r["id"])
            self._docs.append(r["doc"])
            self._metas.append(r["meta"])
    self.dim = int(self._emb.shape[1]) if self._emb.size else 0
    return True
```

```
# ----- 接口
```

```
def add(self, ids, docs, metas, embeddings) -> None:
    emb = np.asarray(embeddings, dtype=np.float32)
    if self._emb.size == 0:
        self._emb = emb
```

```
    else:
        self._emb = np.vstack([self._emb, emb])
    self._ids += list(ids)
    self._docs += list(docs)
    self._metas += [dict(m) for m in metas]
    self.dim = int(self._emb.shape[1])
    self.save()

def query(self, embedding: np.ndarray, top_k: int = 8) -> list[Hit]:
    if self._emb.size == 0:
        return []
    q = np.asarray(embedding, dtype=np.float32).reshape(-1)
    qn = np.linalg.norm(q) or 1.0
    sims = (self._emb @ q) / (np.linalg.norm(self._emb, axis=1) * qn + 1e-12)
    k = min(top_k, len(sims))
    idx = np.argpartition(-sims, k - 1)[:k]
    idx = idx[np.argsort(-sims[idx])]
    return [Hit(self._ids[i], self._docs[i], self._metas[i], float(sims[i]))
            for i in idx]

def count(self) -> int:
    return len(self._ids)

class ChromaStore(BaseStore):
    """chromadb 持久化集合（手册指定的后端）。"""

    backend = "chromadb"

    def __init__(self, path: str | Path, collection: str = "legal"):
        import chromadb
        from chromadb.config import Settings as ChromaSettings

        self.path = Path(path)
        self.path.mkdir(parents=True, exist_ok=True)
        self.client = chromadb.PersistentClient(
            path=str(self.path),
            settings=ChromaSettings(anonymized_telemetry=False, allow_reset=True),
        )
        self.col = self.client.get_or_create_collection(
            name=collection, metadata={"hnsw:space": "cosine"})
        self.dim = 0

    def add(self, ids, docs, metas, embeddings) -> None:
        emb = np.asarray(embeddings, dtype=np.float32)
        self.dim = int(emb.shape[1]) if emb.size else 0
        # chromadb 元数据不接受 None, 需清洗
        clean = [{"k": ("" if v is None else v) for k, v in m.items()} for m in metas]
        # chromadb 单次 upsert 有批量上限（见 InternalError: batch > 5461），
        # 数据集变大（真实文书使文档数到数万）后必须分批写入。
```

```

CHROMA_MAX_BATCH = 4000
n = len(ids)
for i in range(0, n, CHROMA_MAX_BATCH):
    j = min(i + CHROMA_MAX_BATCH, n)
    self.col.upsert(ids=list(ids[i:j]), documents=list(docs[i:j]),
                    metadatas=clean[i:j], embeddings=emb[i:j].tolist())

def query(self, embedding: np.ndarray, top_k: int = 8) -> list[Hit]:
    q = np.asarray(embedding, dtype=np.float32).reshape(-1)
    r = self.col.query(query_embeddings=q.tolist(), n_results=top_k,
                       include=["documents", "metadatas", "distances"])
    docs = (r.get("documents") or [[]])[0]
    metas = (r.get("metadatas") or [[]])[0]
    dists = (r.get("distances") or [[]])[0]
    ids = (r.get("ids") or [[]])[0]
    return [Hit(i, d, m or {}, 1.0 - float(dist))
            for i, d, m, dist in zip(ids, docs, metas, dists)]

def count(self) -> int:
    return int(self.col.count())

def get_store(path: str | Path, collection: str = "legal",
              prefer: str = "auto") -> BaseStore:
    """优先 chromadb, 失败则 numpy。prefer='numpy' 可强制兜底。"""
    if prefer != "numpy":
        try:
            return ChromaStore(path, collection=collection)
        except Exception as exc: # noqa: BLE001
            print(f"[store] chromadb 不可用 ({type(exc).__name__}: {exc}), 改用 NumpyStore")
    return NumpyStore(path)

# =====
# 文件: lawgate/router.py
# =====
# -*- coding: utf-8 -*-
"""三通道路由 (手册 S3.7) + 消融开关 (手册 S6.6)。
路由顺序:
1. 意图命中且槽位齐 → 通道 B (0 次检索; 查无则降级)
2. 否则草稿 k 个 token → 信号 u → 桶 b →  $\tau_b$ 
3.  $u > \tau_b$  → 通道 C (1 次检索); 否则通道 A (0 次检索)

消融开关 (E2) 全部集中在 ``RouterOptions``, 避免在多处复制路由代码:
A1 disable_channel_b 关闭通道 B (法条/案号题全部走门控)
A2 single_tau 用单全局阈值替代桶级阈值
A3 signal 切换信号 (margin / entropy / variance / neglogp)
A4 k_draft 草稿长度
"""

```

```
from __future__ import annotations

import time
from dataclasses import asdict, dataclass, field
from datetime import date
from pathlib import Path

from lawgate.channel.b_structured import ChannelB
from lawgate.channel.c_semantic import Retriever
from lawgate.channel.llm_base import BaseLLM
from lawgate.gate.calibrate import BUCKETS, classify_bucket, load_taus
from lawgate.gate.complexity import complexity_score
from lawgate.gate.intent import detect_intent
from lawgate.gate.signal import SIGNALS, normalize_signal

@dataclass
class RouterOptions:
    signal: str = "margin"
    k_draft: int = 20
    max_new_tokens: int = 192
    top_k: int = 8
    rerank: bool = True
    disable_channel_b: bool = False
    single_tau: bool = False
    tau_single: float = 0.10
    tau_scale: float = 1.0
    # 门控模式 (E0 实测结论见 gate/complexity.py 模块注释)
    # hybrid      : b2 用神经信号, b1/b3/b4 用确定性复杂度评分 (默认)
    # signal      : 全部桶用神经信号 (E0 红灯, 作为消融对照)
    # complexity  : 全部桶用确定性复杂度评分 (手册风险表的降级方案)
    router_mode: str = "hybrid"
    signal_buckets: tuple = ("b2",)

    def to_dict(self) -> dict:
        return asdict(self)

class LegalGateRouter:
    def __init__(self, llm: BaseLLM, retriever: Retriever | None = None,
                 channel_b: ChannelB | None = None,
                 taus: str | Path | dict | None = None,
                 options: RouterOptions | None = None, db_path: str | None = None,
                 draft_llm: BaseLLM | None = None):
        self.llm = llm
        # 草稿 (门控信号) 来源: 默认与回答模型同一个后端; 回答换成 API 后
        # 由 ``draft_source.build_draft_source`` 传入来源链 (见 docs/deviations.md D30)。
        # 为什么单独一个字段而不是“让 llm 自己想办法”: 换源是**决策**,
        # 必须能被 trace 记录、被开关切换 (---draft-source), 不能藏在后端内部。
        self.draft = draft_llm or llm
```

```

self.retriever = retriever
self.B = channel_b or ChannelB(db_path)
self.opt = options or RouterOptions()
if taus is None:
    self.taus = load_taus()
elif isinstance(taus, (str, Path)):
    self.taus = load_taus(taus)
else:
    self.taus = dict(taus)
self.opt.signal = normalize_signal(self.opt.signal)
if self.opt.signal not in SIGNALS:
    raise ValueError(f"未知信号 {self.opt.signal}, 可选 {list(SIGNALS)}")

# ----- 工具
def _tau_for(self, bucket: str) -> float:
    if self.opt.single_tau:
        base = self.opt.tau_single
    else:
        base = float(self.taus.get(bucket, 0.10))
    return round(base * self.opt.tau_scale, 4)

def description(self) -> dict:
    d = {"router": "LegalGate", "options": self.opt.to_dict(),
        "taus": self.taus, "n_buckets": len(BUCKETS)}
    d.update(self.llm.describe())
    if self.draft is not self.llm:
        desc = getattr(self.draft, "describe", None)
        d["draft"] = desc() if callable(desc) else {"draft_backend": type(self.draft).__name__}
    return d

# ----- 主入口
def answer(self, query: str, history: list | None = None,
            meta: dict | None = None) -> dict:
    """非流式入口（实验/评测一律走这里；与流式版共享同一份决策逻辑）。"""
    out: dict = {}
    for ev in self._answer_events(query, history, meta, stream=False):
        if ev.get("event") == "done":
            out = {"answer": ev["answer"], "trace": ev["trace"]}
            if "provision" in ev:
                out["provision"] = ev["provision"]
    if not out:
        raise RuntimeError("router 未产出 done 事件（不应发生）")
    return out

def answer_stream(self, query: str, history: list | None = None,
                  meta: dict | None = None):
    """流式入口：产出事件字典，供 SSE / UI 增量渲染。

    事件序列：``stage`` * → ``delta`` + → ``done``。``done.answer`` 是**权威全文**
    （逐个 delta 的首尾空白可能与它略有出入，前端应以它收口）。

```

```
``stage`` 事件的 ``stage`` 字段: ``channel_b`` (确定性命中) / ``retrieval``  
(走通道 C 前的检索) / ``generate`` (开始生成, 带 channel/u/tau_b/bucket)。  
"""  
  
yield from self._answer_events(query, history, meta, stream=True)  
  
def _answer_events(self, query: str, history: list | None, meta: dict | None,  
                    stream: bool):  
    """``answer`` 与 ``answer_stream`` 的唯一实现 (决策逻辑只此一份)。  
  
    ``stream=False`` 时不做逐 token 生成, 只把整段答案当成一个 delta 产出——  
    这样两条路径的意图判定、路由、trace 字段、通道 B 的早退条件永远一致,  
    不会出现“界面上走 B、接口里却走 C”这种口径漂移。  
    """  
  
    t0 = time.time()  
    history = history or []  
    meta = meta or {}  
  
    # 评测时点 (时效题需要): 只取“评估日期”这一无害字段  
    as_of = None  
    if isinstance(meta.get("temporal"), dict) and meta["temporal"].get("as_of"):  
        try:  
            as_of = date.fromisoformat(meta["temporal"]["as_of"])  
        except ValueError:  
            as_of = None  
  
    intent = detect_intent(query, history, db_path=self.B.db_path)  
    bucket_hint = meta.get("bucket")  
  
    # ----- 通道 B  
    b_eligible = (intent.hit_provision or intent.hit_case_no  
                  or intent.slots_complete)  
    if not self.opt.disable_channel_b and b_eligible:  
        if intent.slots_complete:  
            res = self.B.run(intent.slots, query, as_of=as_of)  
            if res:  
                tr = res["trace"]  
                bucket = bucket_hint or ("b2" if intent.hit_provision else "b3")  
                tr.update({  
                    "u": None, "tau_b": None, "bucket": bucket,  
                    "slots_inherited": list(intent.slots.inherited),  
                    "decision": f"route=B reason={intent.route_b_reason}",  
                    "latency_ms": round((time.time() - t0) * 1000, 1),  
                    "n_retrieval_calls": 0,  
                    "must_show_warning": bool(res.get("must_show_warning")),  
                    "intent": intent.to_dict(),  
                    # 通道 B 是查库拼装, 根本没取草稿: 字段留着并写明原因,  
                    # 免得看 trace 的人以为“门控被跳过了却不知道为什么”  
                    "draft_source": "channel_b (未取草稿)",  
                    "draft_attempts": [],
```

```

    })
    answer = res["answer"]
    yield {"event": "stage", "stage": "channel_b", "channel": "B",
           "bucket": bucket, "decision": tr["decision"]}
    # 通道 B 是查库拼装, 没有 token 流: 整段产出一段 (诚实, 不故弄玄虚)
    yield {"event": "delta", "text": answer}
    done_ev: dict = {"event": "done", "answer": answer, "trace": tr}
    if res.get("provision"):
        done_ev["provision"] = res["provision"]
    yield done_ev
    return

# ----- 门控
stats = self.draft.draft_logprobs(query, history, k=self.opt.k_draft)
u_signal = float(SIGNALS[self.opt.signal](stats))

bucket = bucket_hint if bucket_hint in BUCKETS else classify_bucket({
    "category": "case" if intent.hit_case_no else (
        "provision" if intent.slots.article_no else "concept"),
    "history": history,
    "slots": intent.slots.to_dict(),
    "turn_id": len(history),
})

cplx = complexity_score(query, intent.slots, history,
                        category=meta.get("category"))
u_cplx = cplx.score

if self.opt.router_mode == "signal":
    u, gate_source = u_signal, "neural_signal"
elif self.opt.router_mode == "complexity":
    u, gate_source = u_cplx, "complexity"
else: # hybrid
    if bucket in tuple(self.opt.signal_buckets):
        u, gate_source = u_signal, "neural_signal"
    else:
        u, gate_source = u_cplx, "complexity"

tau_b = self._tau_for(bucket)

retr_info = None
context = None
if u > tau_b:
    channel, n_calls = "C", 1
    # 先播报“要去检索了”: 检索 + 生成在 CPU 上合计十几秒, 静默等待就是黑屏
    yield {"event": "stage", "stage": "retrieval", "channel": "C",
           "bucket": bucket, "u": round(u, 6), "tau_b": tau_b,
           "decision": f"[{gate_source}] u={u:.4f} > tau_{bucket}={tau_b} → C"}
    rr = self.retriever.retrieve(query, top_k=self.opt.top_k,
                                rerank=self.opt.rerank) if self.retriever else None

```

```

        context = rr.context if rr else None
        retr_info = rr.to_dict() if rr else None
    else:
        channel, n_calls = "A", 0

    yield {"event": "stage", "stage": "generate", "channel": channel,
          "bucket": bucket, "u": round(u, 6), "u_signal": round(u_signal, 6),
          "u_complexity": round(u_cplx, 6), "tau_b": tau_b,
          "gate_source": gate_source,
          "decision": (f"[{gate_source}] u={u:.4f} {'>' if u > tau_b else '<='} "
                      f"tau_{bucket}={tau_b} → {channel}")})

    if stream:
        pieces: list[str] = []
        for piece in self.llm.generate_stream(query, history, context=context,
                                              max_tokens=self.opt.max_new_tokens):
            if not piece:
                continue
            pieces.append(piece)
            yield {"event": "delta", "text": piece}
        answer = "".join(pieces).strip()
    else:
        answer = self.llm.generate(query, history, context=context,
                                   max_tokens=self.opt.max_new_tokens)
        yield {"event": "delta", "text": answer}

    trace = {
        "channel": channel,
        "u": round(u, 6),
        "u_signal": round(u_signal, 6),
        "u_complexity": round(u_cplx, 6),
        "gate_source": gate_source,
        "router_mode": self.opt.router_mode,
        "complexity_features": cplx.to_dict()["features"],
        "tau_b": tau_b,
        "bucket": bucket,
        "slots": intent.slots.to_dict(),
        "slots_inherited": list(intent.slots.inherited),
        "validity_status": None,
        "case_verify": None,
        "source_url": None,
        "decision": (f"[{gate_source}] u={u:.4f} {'>' if u > tau_b else '<='} "
                    f"tau_{bucket}={tau_b} → {channel}"),
        "latency_ms": round((time.time() - t0) * 1000, 1),
        "n_retrieval_calls": n_calls,
        "signal": self.opt.signal,
        "k_draft": self.opt.k_draft,
        "intent": intent.to_dict(),
        "retrieval": retr_info,
    }

```

```
        "llm_backend": self.llm.describe().get("llm_backend"),
        # 门控草稿是谁给的 (api / local / rule) + 换源过程 (D30):
        # 没有这两项, 事后无法判断"这条 u 是不是换了模型之后口径变了"。
        "draft_source": getattr(stats, "source", "") or "answer_model",
        "draft_attempts": list(getattr(stats, "attempts", []) or []),
        "draft_seconds": round(float(getattr(stats, "seconds", 0.0) or 0.0), 3),
        "draft_text": stats.text[:200],
    }
    yield {"event": "done", "answer": answer, "trace": trace}

# ----- 便利
def answer_many(self, items: list[dict], as_of_key: str = "temporal") -> list[dict]:
    out = []
    for it in items:
        out.append(self.answer(it.get("query", ""), it.get("history"),
                               meta=it))
    return out

def close(self) -> None:
    self.B.close()
```